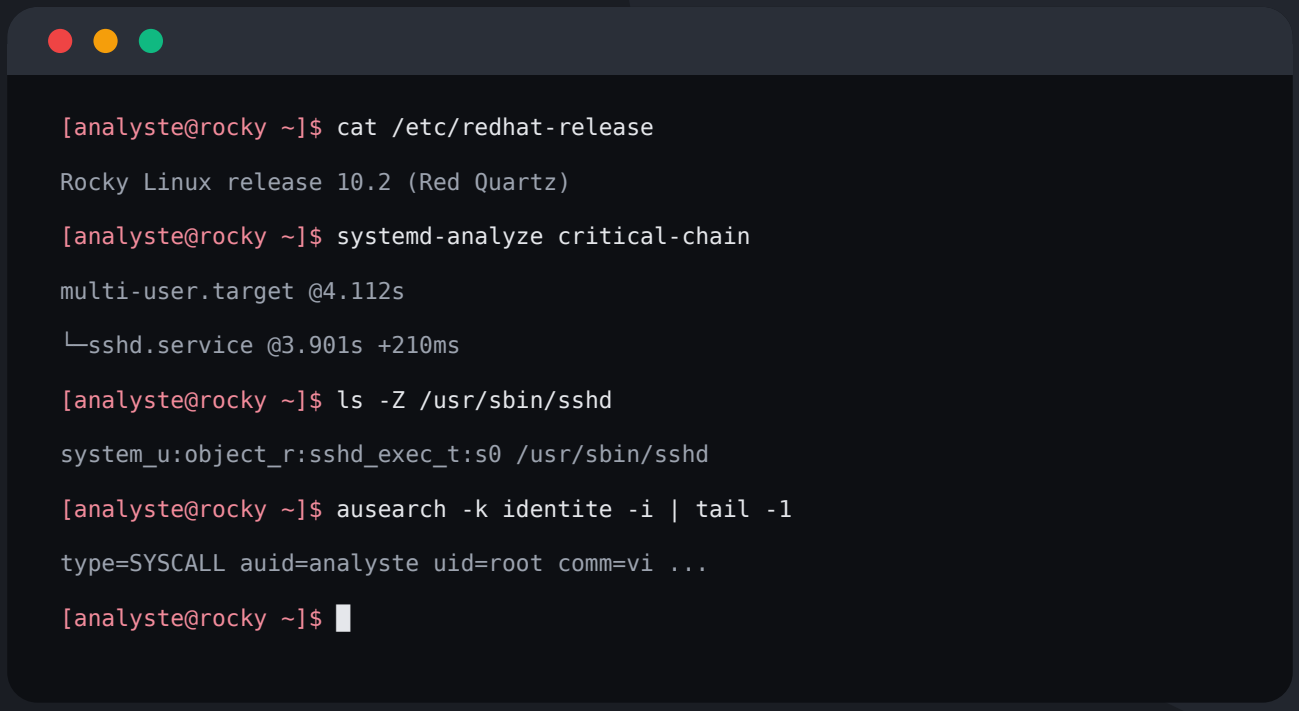


GUIDE TECHNIQUE • NIVEAU EXPERT

Rocky Linux & RHEL

Comprendre le système de l'intérieur

Démarrage, systemd, noyau, RPM/DNF, stockage, réseau,
identités, SELinux, audit, conteneurs : le pourquoi du comment.

Édition EL10 (RHEL 10.2, Rocky Linux 10.2), avec les repères EL8 et EL9.
Chaque chapitre : l'idée clé, le mécanisme, les commandes pour l'observer,
et un regard « analyste SOC » sur ce que cela change pour la détection.

Sources : documentation Red Hat Enterprise Linux 10, Rocky Linux,
pages de manuel Linux, documentation du noyau et des projets amont (voir annexe).

Édition octobre 2026 — RHEL 10.2 / Rocky Linux 10.2

Diffusion libre

Sommaire

Avant de commencer . . . 3

Partie 1 — Le socle : d'où vient le système et comment il est organisé 4

1. La famille Enterprise Linux . . . 5
2. Architecture : noyau, espace utilisateur et arborescence . . . 7

Partie 2 — Le démarrage et systemd 11

3. Le démarrage, maillon par maillon . . . 12
4. systemd : le chef d'orchestre . . . 14

Partie 3 — Processus, mémoire et noyau 18

5. Les processus . . . 19
6. Le noyau : modules, paramètres et observation . . . 22

Partie 4 — Les logiciels : RPM et DNF 24

7. RPM : le paquet et sa base de données . . . 25
8. DNF : résoudre, télécharger, transacter . . . 27

Partie 5 — Le stockage 30

9. Disques, LVM, XFS et montage . . . 31

Partie 6 — Le réseau 34

10. Réseau, pare-feu et temps . . . 35

Partie 7 — Identités, contrôle d'accès et traçabilité 38

11. Utilisateurs, NSS, PAM et sudo . . . 39
12. SELinux : le contrôle d'accès obligatoire . . . 41
13. Cryptographie système et OpenSSH . . . 43
14. Audit et journalisation : la mémoire du système . . . 44

Partie 8 — Conteneurs, mode image et diagnostic 46

15. Podman, conteneurs et mode image . . . 47
16. Diagnostiquer et observer . . . 49
17. Tout assembler : anatomie d'une connexion SSH . . . 50

Annexes 52

Annexe A — Ce que disent les vieux livres, et ce qu'il faut faire aujourd'hui . . . 53
Annexe B — Carte des artefacts pour l'analyste . . . 53
Annexe C — Sources et lectures complémentaires . . . 55

Avant de commencer

> Vous utilisez Rocky Linux tous les jours. Ce guide ne vous apprend pas à taper des commandes : il explique pourquoi le système est conçu ainsi, pour que chaque commande devienne une évidence plutôt qu'un réflexe mémorisé.

Les livres de référence sur Linux datent souvent d'une époque où l'on parlait d'*init* SysV, d'*ifcfg*, d'*iptables* et de *yum*. Une partie de ce savoir reste vraie (le noyau, les processus, les permissions), une autre est devenue fausse. Ce document est écrit pour la génération **EL10** (RHEL 10.2 et Rocky Linux 10.2, publiées en mai 2026) et signale systématiquement ce qui a changé depuis EL8 et EL9.

Comment lire ce guide

Élément	À quoi il sert
L'idée en une phrase	En tête de chaque chapitre : si vous ne retenez qu'une chose, c'est celle-là.
Pourquoi ?	La raison de conception : le problème que le mécanisme résout.
Sous le capot	Le détail interne, pour ceux qui veulent aller au fond des choses.
Regard SOC	Ce que le mécanisme change pour la détection, l'investigation et la réponse à incident : traces laissées, abus possibles, angles morts.
Ce qui a changé	Les différences entre EL8, EL9 et EL10 qui rendent obsolètes les anciens livres.
Encadrés sombres	Commandes à lancer pour **observer** le mécanisme sur votre propre machine. Elles sont en lecture seule sauf mention contraire.
À retenir	La synthèse en fin de chapitre, idéale pour une relecture rapide.

Les chapitres sont indépendants : vous pouvez lire celui dont vous avez besoin. Pour une lecture complète, l'ordre suit la vie d'une machine : elle démarre, lance ses services, exécute des processus, installe des logiciels, stocke, communique, authentifie, contrôle et trace.

> **Une convention**
> Dans ce guide, **EL** (Enterprise Linux) désigne toute la famille : RHEL, Rocky Linux, AlmaLinux, Oracle Linux et CentOS Stream. Ce qui est dit pour RHEL 10 vaut pour Rocky Linux 10, sauf mention contraire (dépôts, abonnements, outils propres à Red Hat).

PARTIE 1

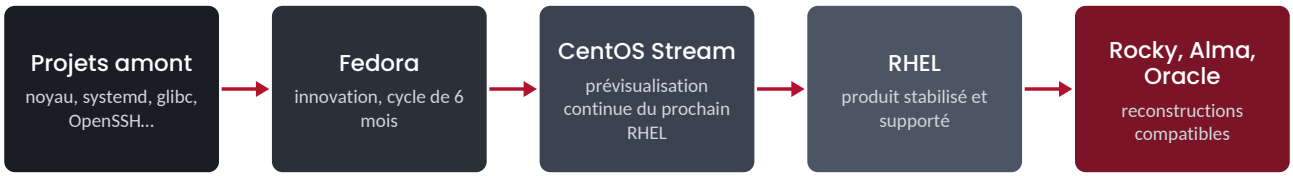
Le socle : d’où vient le système et comment il est organisé

Avant de démonter le moteur, il faut savoir d’où il sort. Ce que vous appelez « Rocky Linux » est le dernier maillon d’une chaîne de distributions, et cette généalogie explique presque tout : le rythme des versions, les numéros de noyau trompeurs et la philosophie de stabilité.

1. La famille Enterprise Linux

> RHEL n'est pas « un Linux figé » : c'est un système dont l'interface est figée pendant dix ans, pendant que l'intérieur continue d'évoluer par rétroportage.

La chaîne de fabrication



Chaque version majeure de RHEL est « branchée » depuis une version de Fedora, puis développée publiquement dans CentOS Stream.

Une nouvelle technologie naît dans les projets amont, est intégrée et testée dans Fedora, puis mûrit dans CentOS Stream qui préfigure la prochaine version mineure de RHEL. Red Hat publie RHEL, et des projets comme Rocky Linux le reconstruisent à partir des sources pour obtenir un système compatible « bogue pour bogue ».

Ce qui a changé — l'accès aux sources

Depuis juin 2023, Red Hat ne publie plus librement les sources de RHEL sur le dépôt git de CentOS. Rocky Linux a annoncé les reconstituer à partir de plusieurs sources : CentOS Stream, les paquets amont, et les RPM sources obtenus légalement via les images de conteneur UBI et des instances cloud facturées à l'usage. Pour vous, rien ne change à l'usage ; cela explique en revanche le léger décalage entre une sortie de RHEL et celle de Rocky (8 à 22 jours sur les dernières versions).

Le rythme des versions

Depuis RHEL 8, une version majeure sort tous les trois ans (8 en 2019, 9 en 2022, 10 en 2025) et une version mineure tous les six mois. RHEL 10 est sortie le 20 mai 2025 ; Rocky Linux 10.0 « Red Quartz » a suivi le 11 juin 2025, et Rocky Linux 10.2 le 28 mai 2026. Rocky Linux 10 bénéficie d'un support général jusqu'au 31 mai 2030 et de correctifs de sécurité jusqu'au 31 mai 2035.

Pourquoi ? — dix ans de stabilité

Un éditeur de logiciel métier certifie son application sur « RHEL 10 ». Il a besoin que les bibliothèques et les interfaces sur lesquelles il s'appuie ne changent pas pendant toute la vie du système. Red Hat publie pour cela un guide de compatibilité applicative qui classe les composants par niveau de garantie : certaines ABI sont gelées pour dix ans, d'autres peuvent évoluer entre versions mineures. C'est le contrat qui justifie le prix de RHEL… et dont les reconstructions héritent.

Le paradoxe du numéro de noyau

Rocky Linux 10.2 tourne avec un noyau `6.12.0-211.16.1.el10_2`. La partie `6.12.0` ne bouge plus pendant toute la vie d'EL10 ; seule la partie après le tiret (la *release*) augmente. Pourtant, ce noyau contient des milliers de correctifs et de fonctionnalités issus de noyaux amont bien plus récents : Red Hat les **rétroporte** (backport) en préservant l'ABI du noyau (kABI) dont dépendent les pilotes tiers.

Regard SOC — faux positifs des scanners

Un scanner qui compare « 6.12.0 » à la version amont corrigeant une CVE conclura à tort que le système est vulnérable. La seule source fiable est l'avis de l'éditeur (RHSA chez Red Hat, RLSA chez Rocky) ou le journal des modifications du paquet : `rpm -q --changelog kernel | grep CVE-2026`. Inversement, une version amont « récente » compilée à la main sur un serveur EL échappe à tout suivi d'errata : c'est un signal à investiguer.

EL8, EL9, EL10 : ce qui a bougé

Composant	EL8 (2019)	EL9 (2022)	EL10 (2025)
Noyau de base	4.18	5.14	6.12
Niveau x86-64 minimal	x86-64 (v1)	x86-64-v2	x86-64-v3
Gestion des paquets	DNF 4 + modules	DNF 4 + modules	DNF 4, sans modularité
Configuration réseau	ifcfg par défaut	keyfile par défaut, ifcfg toléré	keyfile uniquement
Python système	3.6	3.9	3.12
OpenSSH	8.0	8.7	9.9
Serveur graphique	Xorg + Wayland	Wayland, Xorg déprécié	Wayland seul (Xwayland pour X11)
Bureau à distance	VNC	VNC	RDP
Runtime de conteneurs	runc	crun (runc disponible)	crun (runc retiré)
Réseau des conteneurs	CNI	netavark (CNI toléré)	netavark (CNI retiré)
Agrégation réseau	team ou bond	team ou bond	bond uniquement (teamd retiré)
Client DHCP	dhclient ou interne	interne à NetworkManager	interne (dhclient retiré)

x86-64-v3 : pourquoi votre vieux serveur ne démarre plus EL10

Les « niveaux » x86-64 regroupent des extensions du jeu d'instructions : v2 ajoute notamment SSE4.2 et POPCNT (processeurs d'environ 2009), v3 ajoute AVX2, BMI1/BMI2, FMA et MOVBE (Intel Haswell, 2013, et équivalents AMD). En compilant tout le système pour v3, Red Hat gagne en performance (chiffrement, compression, calcul vectoriel), au prix de l'exclusion des processeurs plus anciens. AlmaLinux est la seule reconstruction à publier une variante v2 d'EL10.

```
observer


# Quels niveaux x86-64 votre processeur et votre glibc supportent-ils ?
/lib64/ld-linux-x86-64.so.2 --help | grep -A4 "Subdirectories of glibc-hwcaps"
#   x86-64-v4
#   x86-64-v3 (supported, searched)
#   x86-64-v2 (supported, searched)

# Identifier précisément le système
cat /etc/os-release          # norme freedesktop, à privilégier dans les scripts
rpm -q --whatprovides redhat-release
uname -r                     # 6.12.0-211.16.1.el10_2.x86_64
```

Les dépôts et leur logique

Dépôt	Contenu	Garantie
BaseOS	Le cœur du système : noyau, systemd, glibc, outils de base.	Cycle de vie complet de la version majeure.
AppStream	Applications, langages, bases de données, serveurs web.	Cycle de vie propre à chaque composant (parfois plus court).
CRB	« CodeReady Builder » : en-têtes et outils de compilation (désactivé par défaut).	Non supporté pour la production chez Red Hat.
extras (Rocky)	Paquets propres à Rocky, dont `epel-release`.	Projet Rocky.
EPEL	Paquets communautaires Fedora reconstruits pour EL.	Communautaire, aucune garantie éditeur.

Ce qui a changé — la fin de la modularité

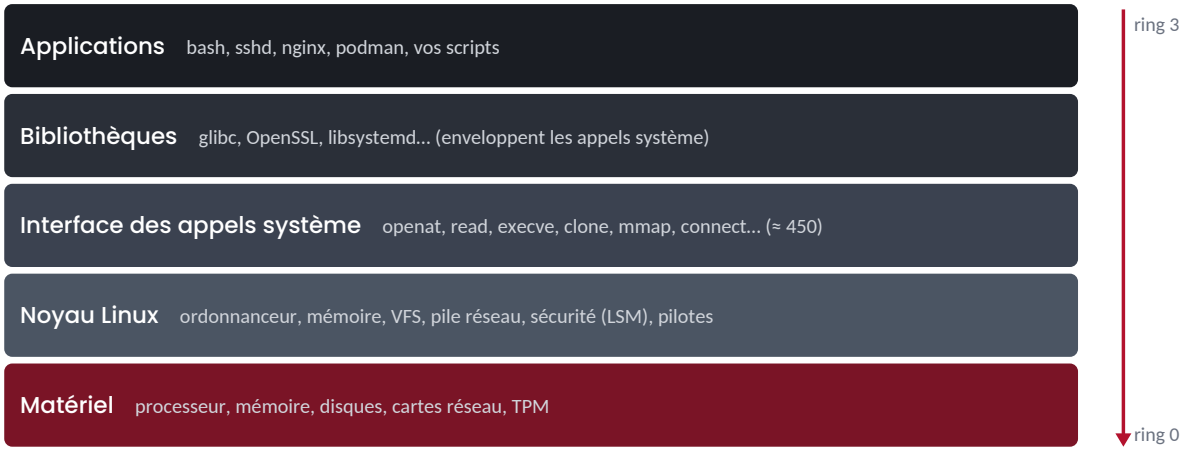
EL8 et EL9 proposaient des « modules » (`dnf module enable nodejs:20`) pour choisir une version d'un logiciel. Red Hat n'en livre plus aucun dans RHEL 10 : les versions alternatives arrivent comme des paquets ordinaires. Les tutoriels qui utilisent `dnf module` sont donc obsolètes pour EL10.

À retenir

- Fedora → CentOS Stream → RHEL → Rocky : chaque étape stabilise la précédente.
- Le numéro de version amont (6.12) est figé ; seule la *release* évolue. Évaluez les vulnérabilités avec les errata, jamais avec le numéro amont.
- EL10 exige un processeur x86-64-v3 et abandonne la modularité, ifcfg, Xorg, teamd, dhclient, runc et CNI.

2. Architecture : noyau, espace utilisateur et arborescence

> Tout programme vit dans une cage (l'espace utilisateur) et ne peut toucher au matériel qu'en demandant poliment au noyau, par un appel système.



Les trois couches du haut s'exécutent en espace utilisateur (ring 3) ; le noyau, en mode privilégié (ring 0).

Deux mondes séparés par le processeur

Le processeur dispose de niveaux de privilège. Le noyau s'exécute au niveau le plus privilégié et peut tout faire ; les programmes s'exécutent au niveau le moins privilégié et ne peuvent ni accéder directement au matériel, ni lire la mémoire d'un autre processus. Pour ouvrir un fichier, un programme exécute une instruction spéciale qui bascule le processeur en mode noyau : c'est l'**appel système**. Le noyau vérifie les droits, fait le travail, puis rend la main.

```
observer

# Voir les appels système d'une commande (paquet strace)
strace -f -e trace=openat,execve,connect -o /tmp/trace.txt \
       curl -s https://example.org >/dev/null
grep -E "execve|connect" /tmp/trace.txt | head

# Résumé statistique : quels appels, combien de fois, combien de temps
strace -c ls /etc >/dev/null
```

« Tout est fichier » et le VFS

Le noyau présente presque tout comme des fichiers grâce à une couche d'abstraction, le **VFS** (Virtual File System). Un même `read()` fonctionne sur un fichier XFS, un tube, une socket, un périphérique ou un pseudo-fichier du noyau. Trois systèmes de fichiers virtuels sont essentiels :

- `/proc` : une fenêtre sur les processus (`/proc/1234/`) et des réglages du noyau (`/proc/sys/`).
- `/sys` (sysfs) : l'arbre des périphériques et des pilotes tel que le noyau le voit.
- `/dev` : les périphériques, peuplé par le noyau (devtmpfs) puis complété par **udev** (règles, liens symboliques stables comme `/dev/disk/by-uuid/`).

Une arborescence qui a changé : la fusion de /usr

Sur un système EL, `/bin`, `/sbin`, `/lib` et `/lib64` ne sont plus des répertoires : ce sont des liens symboliques vers `/usr/bin`, `/usr/sbin`, `/usr/lib` et `/usr/lib64` (depuis EL7).

> **Pourquoi ? — tout mettre dans /usr**
>
> La séparation historique entre `/bin` et `/usr/bin` venait de disques trop petits dans les années 1970. En regroupant tout le logiciel fourni par l'éditeur sous `/usr`, ce répertoire devient un bloc cohérent que l'on peut monter en lecture seule, partager, ou remplacer d'un coup. C'est la condition technique du mode image (chapitre 15), où `/usr` est immuable.

Le principe le plus utile du système : trois niveaux de configuration

Une grande partie des composants modernes (systemd, sysctl, modprobe, udev, tmpfiles, journald, crypto-policies…) applique la même règle de précédence. Une fois comprise, elle explique des dizaines de comportements.

/usr/lib/…	/run/…	/etc/…
Valeurs de l'éditeur. Écrasées à chaque mise à jour. Ne jamais modifier.	Valeurs temporaires. Générées au démarrage. Perdues au redémarrage.	Vos choix d'administrateur. Jamais écrasés par une mise à jour. Priorité la plus haute.

Un fichier de même nom dans /etc masque celui de /usr/lib ; un fichier « .d/.conf » s'ajoute par-dessus.*

Exemple : le fichier d'unité de sshd est livré dans `/usr/lib/systemd/system/sshd.service`. Pour le modifier, on ne l'édite pas : on crée une surcharge dans `/etc/systemd/system/sshd.service.d/` (avec `systemctl edit`

`sshd`). La mise à jour du paquet remplacera le fichier de l'éditeur, et votre surcharge restera appliquée.

Ce qui a changé — des fichiers absents de /etc

Avec les versions récentes de systemd, une partie des fichiers de configuration par défaut n'est plus livrée dans `/etc` mais uniquement sous `/usr/lib`. Sur EL10, Red Hat le signale notamment pour journald : on configure alors par un fichier de surcharge dans `/etc/systemd/journald.conf.d/`. Ne concluez pas qu'un service « n'a pas de configuration » parce que son fichier manque dans `/etc`.

Sous le capot — rpmnew et rpmsave

Les fichiers de configuration livrés par RPM sont marqués `%config(noreplace)`. Si vous les avez modifiés, une mise à jour ne les écrase pas : la nouvelle version est déposée à côté sous le nom `fichier.rpmnew`. À la désinstallation, votre version modifiée est conservée en `.rpmsave`. Une recherche régulière de ces fichiers révèle les configurations qui divergent de l'éditeur.

```
observer

# Voir la configuration effective d'une unité, surcharges comprises
systemctl cat sshd

# Lister les surcharges d'unités systemd présentes sur la machine
systemd-delta --type=overridden,extended

# Configurations qui ont divergé lors de mises à jour
find /etc -name "*.rpmnew" -o -name "*.rpmsave"
```

Regard SOC — où se cachent les modifications

Un attaquant qui veut persister sans modifier de binaire préfère les répertoires `.d/` et `/etc` : une surcharge de `sshd.service`, un fichier dans `/etc/profile.d/`, `/etc/ld.so.preload`, une règle udev qui exécute un programme. `systemd-delta` et la comparaison avec `rpm -V` (chapitre 7) donnent une vue rapide de tout ce qui s'écarte de l'état livré par l'éditeur.

Les répertoires et leur sens

Chemin	Rôle	Particularité
/etc	Configuration de la machine	Petit, texte, à sauvegarder et à surveiller.
/usr	Logiciel livré par les paquets	Reconstitutible à partir des paquets ; immuable en mode image.
/var	Données variables : journaux, bases, caches, files	Persistant ; contient `/var/log` et `/var/lib`.
/run	État d'exécution (PID, sockets)	tmpfs : vidé à chaque démarrage.
/tmp	Fichiers temporaires	Nettoyé périodiquement par systemd-tmpfiles.
/dev/shm	Mémoire partagée POSIX	tmpfs inscriptible par tous : lieu classique de dépôt de charges malveillantes.
/boot	Noyaux, initramfs, configuration GRUB	Souvent une partition séparée sur une installation classique.
/opt, /srv	Logiciels tiers, données servies	Conventionnels, peu utilisés par les paquets EL.

À retenir

- Les programmes n'accèdent au matériel que via les appels système ; `strace` permet de les voir.
- `/bin` → `/usr/bin` : tout le logiciel de l'éditeur vit sous `/usr`.
- Règle des trois niveaux : `/usr/lib` (éditeur) < `/run` (temporaire) < `/etc` (vous).
- Ne modifiez jamais un fichier de `/usr/lib` : surchargez-le dans `/etc`.

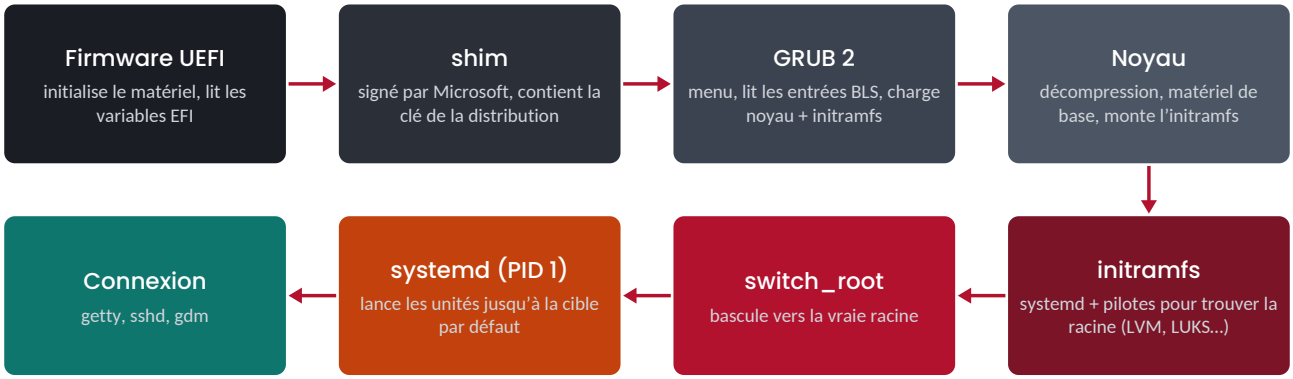
PARTIE 2

Le démarrage et systemd

Entre l'appui sur le bouton et l'invite de connexion, une dizaine de programmes se passent le relais, chacun vérifiant le suivant. Comprendre cette chaîne, c'est savoir réparer un serveur qui ne démarre plus, et savoir où un attaquant peut s'insérer.

3. Le démarrage, maillon par maillon

> Le démarrage est une course de relais : chaque étape charge la suivante, et avec Secure Boot, chaque étape vérifie la signature de la suivante avant de lui passer la main.



La chaîne de démarrage UEFI d'un système EL10.

Étape 1 : le firmware UEFI et Secure Boot

Le firmware UEFI lit dans sa mémoire non volatile (NVRAM) l'ordre de démarrage et le chemin d'un fichier exécutable sur la partition système EFI (ESP, montée sur `/boot/efi`). Avec Secure Boot actif, il n'exécute que des binaires signés par une clé présente dans sa base de confiance, qui contient en pratique la clé de Microsoft.

> **Pourquoi ? — shim existe**
>
> Les distributions ne peuvent pas faire signer chaque version de GRUB par Microsoft. Elles font donc signer une seule fois un petit chargeur, **shim**, qui embarque leur propre clé publique. Le firmware fait confiance à shim (signé Microsoft) ; shim fait confiance à GRUB et au noyau (signés par Red Hat ou par Rocky). Une base supplémentaire, MOK (Machine Owner Key), gérée par `mokutil`, permet d'ajouter vos propres clés, par exemple pour un pilote compilé localement.

Étape 2 : GRUB 2 et les entrées BLS

GRUB affiche le menu et charge le noyau et l'initramfs. Sur EL, chaque noyau installé possède son propre petit fichier d'entrée au format **BLS** (Boot Loader Specification) dans `/boot/loader/entries/`, nommé d'après l'identifiant de machine et la version du noyau. Le fichier `/boot/grub2/grub.cfg` ne liste plus les noyaux : il lit ces entrées.

> **Pourquoi ? — les entrées BLS**
>
> Autrefois, installer un noyau imposait de régénérer tout `grub.cfg` avec un script fragile. Avec BLS, le paquet du noyau dépose simplement son fichier d'entrée, et la suppression du paquet le retire. L'outil `grubby` modifie les paramètres de manière fiable, pour un noyau ou pour tous.

```
observer


ls /boot/loader/entries/                    # une entrée par noyau installé
grubby --default-kernel                     # noyau démarré par défaut
grubby --info=ALL | grep -E "^kernel|^args"  # paramètres de chaque entrée

# Ajouter un paramètre noyau à toutes les entrées (persistant et appliqué aux futurs noyaux)
grubby --update-kernel=ALL --args="audit=1 audit_backlog_limit=8192"
# paramètres réellement utilisés au démarrage courant
cat /proc/cmdline
mokutil --sb-state                          # Secure Boot actif ?
```

Étape 3 : le noyau et l'initramfs

Le noyau se décompresse, initialise le processeur, la mémoire et les pilotes intégrés. Mais il a un problème de poule et d'œuf : la racine `/` peut se trouver sur un volume LVM, chiffré par LUKS, sur un RAID ou en iSCSI, et les pilotes nécessaires sont des modules… stockés sur cette même racine. L'**initramfs** résout ce problème : c'est une archive chargée en mémoire par GRUB, qui contient un mini-système avec systemd, udev et juste les modules et outils nécessaires.

> **Sous le capot — dracut**
> L'initramfs est généré par **dracut** lors de l'installation de chaque noyau, dans `/boot/initramfs-$(uname -r).img`. dracut analyse la machine pour n'inclure que l'utile (mode « hostonly »). Après un changement de stockage (nouveau pilote, chiffrement), il faut parfois le régénérer avec `dracut -f`. `lsinitrd` affiche son contenu.

Une fois la vraie racine trouvée, déverrouillée et montée sur `/sysroot`, l'initramfs exécute `switch_root` : la racine en mémoire est abandonnée, `/sysroot` devient `/`, et systemd se ré-exécute depuis le vrai système.

Quand le démarrage échoue

Mode	Comment	Ce que vous obtenez
rescue.target	Ajouter `systemd.unit=rescue.target` dans GRUB (touche e)	Mono-utilisateur, systèmes de fichiers montés, peu de services. Mot de passe root requis.
emergency.target	`systemd.unit=emergency.target`	Racine montée en lecture seule, presque rien d'autre. Pour un fstab cassé.
rd.break	Ajouter `rd.break`	Shell dans l'initramfs, avant le basculement : la vraie racine est en `/sysroot`, en lecture seule.

> **Regard SOC — l'accès physique**
> La procédure classique de réinitialisation du mot de passe root (`rd.break`, remontage de `/sysroot` en écriture, `chroot`, `passwd`, `touch /.autorelabel`) montre qu'un accès à la console équivaut à un accès root. Les seules parades sont un mot de passe GRUB (`grub2-setpassword`) et surtout le chiffrement du disque. À surveiller : modifications de `/boot/loader/entries/`, des paramètres noyau (ajout de `selinux=0` ou `init=`), et l'état de Secure Boot.

À retenir

- UEFI → shim → GRUB → noyau → initramfs → switch_root → systemd.
- Les noyaux sont décrits par des entrées BLS ; on les modifie avec `grubby`, jamais à la main dans `grub.cfg`.
- L'initramfs existe parce que les pilotes nécessaires pour trouver la racine sont sur la racine.
- Accès console + disque non chiffré = accès root.

4. systemd : le chef d'orchestre

> systemd ne lance pas des scripts dans l'ordre : il construit un graphe de dépendances entre des unités et démarre en parallèle tout ce qui peut l'être.

systemd est le processus numéro 1. Il démarre et surveille les services, mais aussi les montages, les sockets, les timers, les périphériques et les cgroups. Il remplace le vieux script d'init qui exécutait des scripts shell un par un, lentement et sans savoir si un service avait réellement démarré.

Les unités

Type	Rôle	Exemple
.service	Un processus (démon ou tâche ponctuelle)	sshd.service
.socket	Un point d'écoute qui démarre un service à la première connexion	cockpit.socket
.target	Un point de synchronisation, un « état » du système	multi-user.target
.mount / .automount	Un point de montage (souvent généré depuis fstab)	var-log.mount
.timer	Une planification qui déclenche un service	dnf-makecache.timer
.path	Une surveillance de fichier qui déclenche un service	—
.slice / .scope	Des groupes de processus (cgroups) pour les ressources	user-1000.slice
.device	Un périphérique annoncé par udev	dev-sda.device

La confusion n°1 : dépendance et ordre sont deux choses différentes

C'est la notion la plus mal comprise de systemd, et la source de nombreux services qui « démarrent parfois trop tôt ».

Wants= / Requires=

Disent QUOI démarrer avec l'unité.
Requires : si la dépendance échoue ou s'arrête, l'unité aussi.
Wants : démarrage souhaité, échec toléré.
N'imposent AUCUN ordre.

After= / Before=

Disent DANS QUEL ORDRE.
After=B : attendre que B ait fini de démarrer.
Ne démarrent RIEN par eux-mêmes.
Sans eux, tout part en parallèle.

Pour « démarrer B avant A et attendre », il faut les deux : Wants=B et After=B.

Pourquoi ? — cette séparation

Elle permet le parallélisme maximal. Si chaque dépendance impliquait un ordre, le démarrage redeviendrait séquentiel. systemd préfère que vous exprimiez séparément « j'ai besoin de » et « après ». C'est aussi pour cela que `network.target` ne garantit pas un réseau opérationnel : pour attendre une adresse IP, il faut `Wants=network-online.target` et `After=network-online.target`.

Anatomie d'un service

unité de service

```
# /etc/systemd/system/collecteur.service  (exemple commenté)
[Unit]
Description=Collecteur de journaux maison
Wants=network-online.target
After=network-online.target

[Service]
Type=notify                    # le programme signale lui-même qu'il est prêt (sd_notify)
User=collecteur                # jamais root si ce n'est pas nécessaire
ExecStart=/usr/local/bin/collecteur --config /etc/collecteur.yml
Restart=on-failure             # relance automatique en cas de plantage
RestartSec=5
# Durcissement : le service ne voit qu'une partie du système
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes
NoNewPrivileges=yes
ReadWritePaths=/var/lib/collecteur

[Install]
# "systemctl enable" crée le lien dans multi-user.target.wants/
WantedBy=multi-user.target
```

Type=	systemd considère le service démarré…	Usage
simple	dès le fork, avant même l'exécution	Défaut historique ; peu fiable pour l'ordre.
exec	après le succès de `execve()`	Meilleur choix pour un programme qui reste au premier plan.
forking	quand le processus parent se termine	Anciens démons qui se « détachent » ; nécessite souvent PIDFile.
notify	quand le programme envoie READY=1	Le plus précis ; utilisé par de nombreux démons modernes.
oneshot	quand la commande est terminée	Tâches ponctuelles (souvent avec RemainAfterExit=yes).

Ce que fait vraiment « systemctl enable »

`systemctl enable sshd` ne démarre rien. Il lit la section `[Install]` et crée un lien symbolique dans `/etc/systemd/system/multi-user.target.wants/`. Au démarrage suivant, quand systemd active `multi-user.target`, il voit ce lien et ajoute sshd au graphe. `systemctl start` démarre immédiatement sans rien rendre persistant ; `enable --now` fait les deux. `mask` pointe l'unité vers `/dev/null` : elle ne peut plus être démarrée, même par dépendance.

Les cibles : les « niveaux d'exécution » modernes

Une cible est un regroupement : `multi-user.target` (serveur), `graphical.target` (bureau), `rescue.target`, `emergency.target`. La cible de démarrage est le lien `/etc/systemd/system/default.target` (`systemctl get-default`). Pendant l'exécution, `systemctl isolate` bascule d'une cible à l'autre.

Les générateurs : quand fstab devient des unités

Au démarrage et à chaque `systemctl daemon-reload`, de petits programmes appelés **générateurs** traduisent des configurations classiques en unités. Le plus connu, `systemd-fstab-generator`, transforme chaque ligne de `/etc/fstab` en unité `.mount` placée dans `/run/systemd/generator/`.

> **Le piège classique**
> Après avoir modifié `/etc/fstab`, lancez `systemctl daemon-reload`. Sinon systemd continue d'utiliser les anciennes unités générées, et un `mount -a` peut sembler fonctionner alors que le prochain démarrage échouera.

Activation par socket et timers

Avec l'activation par socket, systemd ouvre lui-même le port d'écoute et ne démarre le service qu'à la première connexion, en lui transmettant la socket déjà ouverte. Cela accélère le démarrage et supprime les problèmes d'ordre (le client peut se connecter avant que le service soit prêt : la connexion attend dans la file du noyau). Les **timers** remplacent avantageusement cron : journalisés, avec dépendances, aléa de déclenchement (`RandomizedDelaySec`) et rattrapage des exécutions manquées (`Persistent=true`).

```
observer

# durée du démarrage (firmware, chargeur, noyau, espace utilisateur)
systemd-analyze
systemd-analyze blame | head         # unités les plus lentes
systemd-analyze critical-chain       # chemin critique du graphe
systemctl list-dependencies sshd     # arbre des dépendances
systemctl list-timers --all          # toutes les planifications systemd
systemctl --failed                   # unités en échec
# score d'exposition d'un service (0 = très confiné, 10 = exposé)
systemd-analyze security sshd
```

journald : le journal intégré

`systemd-journald` collecte tout ce qui est écrit sur la sortie standard des services, les messages syslog, les messages du noyau et de l'audit, et les enregistre dans un format binaire indexé. Chaque entrée porte des champs fiables ajoutés par journald lui-même (préfixés par _), comme `_PID`, `_UID`, `_COMM`, `_SYSTEMD_UNIT` ou `_BOOT_ID` : un processus peut mentir sur son nom dans un message syslog, pas sur ces champs.

> **Persistance du journal**
> Par défaut sur EL, le journal peut être conservé uniquement en mémoire (`/run/log/journal`) et disparaître au redémarrage ; les journaux texte de `/var/log/messages` sont écrits par rsyslog, qui lit le journal. Pour une conservation sur disque, définissez `Storage=persistent` dans une surcharge `/etc/systemd/journald.conf.d/*.conf`, redémarrez journald puis lancez `journalctl --flush`.

```
observer

journalctl -b -p warning              # avertissements et plus graves depuis le démarrage
journalctl -u sshd --since "1 hour ago"
journalctl _UID=0 _COMM=sudo          # filtrage sur des champs de confiance
journalctl -o json-pretty -n 1        # voir tous les champs d'une entrée
journalctl --list-boots               # démarrages connus (si le journal est persistant)
```

Regard SOC — persistance via systemd

systemd est devenu l'emplacement favori de persistance sous Linux : service ou timer déposé dans `/etc/systemd/system/`, surcharge d'un service légitime (`ExecStartPre=` ajouté), unités utilisateur dans `~/.config/systemd/user/` (avec `loginctl enable-linger` pour survivre à la déconnexion), générateur malveillant dans `/etc/systemd/system-generators/`. À auditer : `systemctl list-unit-files --state=enabled`, `systemctl list-timers --all`, `systemd-delta`, et les dates de modification de ces répertoires.

À retenir

- Wants/Requires = quoi démarrer ; After/Before = dans quel ordre. Il faut souvent les deux.
- `enable` crée un lien dans un répertoire `.wants` ; `start` démarre maintenant ; `mask` interdit.
- Après modification de fstab ou d'une unité : `systemctl daemon-reload`.
- Les champs _PID, _UID, _COMM de journald sont fiables ; le texte du message ne l'est pas.

PARTIE 3

Processus, mémoire et noyau

Un serveur passe sa vie à créer des processus, à leur distribuer du temps processeur et de la mémoire, et à les isoler les uns des autres. Ces mécanismes sont aussi la base des conteneurs et le terrain de jeu des attaquants.

5. Les processus

> Sous Linux, on ne crée jamais un processus « à partir de rien » : on clone un processus existant (fork), puis le clone remplace son programme par un autre (exec).



Pourquoi ? — fork puis exec

Entre les deux appels, l'enfant peut préparer son environnement : rediriger ses entrées-sorties, changer de répertoire, abaisser ses privilèges, entrer dans un namespace. C'est ainsi que le shell implémente `ls > fichier` : il ouvre le fichier dans l'enfant, puis appelle exec. La **copie sur écriture** rend fork peu coûteux : les pages mémoire ne sont réellement dupliquées que si l'un des deux processus les modifie.

L'identité d'un processus

Attribut	Signification	Où le lire
PID / PPID	Identifiant du processus et de son parent	`ps -o pid,ppid,cmd`
UID réel / effectif	Qui a lancé le processus / avec quels droits il agit (diffèrent pour un setuid)	`/proc/PID/status`
loginuid (auid)	L'utilisateur qui s'est authentifié à l'origine de la session ; inchangé par sudo ou su	`/proc/PID/loginuid`
Capacités	Fragments des privilèges de root (CAP_NET_BIND_SERVICE…)	`getpcaps PID`
Contexte SELinux	Domaine dans lequel le processus est confiné	`ps -eZ`
cgroup	Groupe de ressources (service, session, conteneur)	`/proc/PID/cgroup`
Namespaces	Vues isolées du système (réseau, montages, PID…)	`ls -l /proc/PID/ns`

États, signaux et zombies

Un processus est en exécution (**R**), en sommeil interruptible (**S**, il attend un événement), en sommeil non interruptible (**D**, il attend une entrée-sortie disque et ne réagit même pas à SIGKILL), arrêté (**T**) ou zombie (**Z**). Un zombie n'occupe plus de mémoire : c'est une simple entrée qui attend que son parent lise son code retour. Si le parent meurt, ses enfants sont adoptés par systemd (ou par un processus « subreaper »), qui les récolte.

Signal	N°	Effet usuel
SIGTERM	15	Demande polie d'arrêt ; le programme peut nettoyer. Envoyé par défaut par kill et par systemctl stop.
SIGKILL	9	Arrêt immédiat par le noyau, impossible à intercepter. Dernier recours (pas de nettoyage).
SIGHUP	1	Historiquement « terminal raccroché » ; par convention, de nombreux démons relisent leur configuration.
SIGSTOP / SIGCONT	19 / 18	Gèle / reprend un processus. Utile pour figer un processus suspect sans perdre son état.

/proc : l'autopsie d'un processus vivant

```
observer

P=$(pgrep -o sshd)
# binaire réellement exécuté (même s'il a été supprimé ou renommé)
ls -l /proc/$P/exe
# ligne de commande (modifiable par le processus lui-même !)
tr '\0' ' ' < /proc/$P/cmdline; echo
cat /proc/$P/status | grep -E "Uid|Gid|Cap|NoNewPrivs|Seccomp"
# 4294967295 = aucun utilisateur connecté à l'origine (service système)
cat /proc/$P/loginuid
ls -l /proc/$P/fd | head       # fichiers et sockets ouverts
# projections mémoire : bibliothèques chargées, zones anonymes exécutables
cat /proc/$P/maps | head
cat /proc/$P/cgroup            # unité systemd ou conteneur d'appartenance
```

Regard SOC — ce que /proc révèle

Un binaire supprimé après lancement apparaît comme `/proc/PID/exe -> /chemin (deleted)` : on peut encore le copier depuis `/proc/PID/exe` pour l'analyser. Une ligne de commande peut être falsifiée par le processus (`argv[0]` réécrit pour imiter `[kworker]`), mais `exe`, `maps` et le PPID ne mentent pas. Un « thread noyau » entre crochets qui possède un fichier `exe` ou un parent autre que `kthreadd` (PID 2) est un imposteur.

Capacités : découper les pouvoirs de root

Plutôt que « root peut tout », le noyau découpe les privilèges en une quarantaine de **capacités** : lier un port inférieur à 1024 (`CAP_NET_BIND_SERVICE`), administrer le réseau (`CAP_NET_ADMIN`), contourner les permissions de fichiers (`CAP_DAC_OVERRIDE`), etc. Un serveur web peut ainsi écouter sur le port 443 sans être root. Les capacités peuvent être attachées à un fichier exécutable, comme un setuid plus fin.

```
observer

getcap -r /usr/bin /usr/sbin 2>/dev/null     # exécutables dotés de capacités de fichier
find / -xdev -perm /6000 -type f 2>/dev/null # exécutables setuid / setgid
```

La mémoire virtuelle

Chaque processus croit disposer d'un immense espace d'adressage contigu qui lui appartient. En réalité, le noyau et l'unité de gestion mémoire (MMU) du processeur traduisent chaque adresse virtuelle en adresse physique, par pages de 4 Kio. Une page peut être en mémoire, sur le swap, partagée avec d'autres processus (bibliothèques), ou pas encore allouée du tout.

> **Pourquoi ? — « free » semble dire que la mémoire est pleine**
> Linux utilise la mémoire libre comme **cache** des fichiers lus (page cache) : de la mémoire inutilisée est de la mémoire gaspillée. Ce cache est rendu instantanément quand un programme en a besoin. La colonne à regarder dans `free -h` est **available**, pas *free*.

Le noyau accepte par défaut d'accorder plus de mémoire qu'il n'en possède (**overcommit**), parce que les programmes réservent beaucoup plus qu'ils n'utilisent. Si la mémoire réelle vient vraiment à manquer, l'**OOM killer** choisit un processus à tuer selon un score (`/proc/PID/oom_score`), ajustable par `oom_score_adj` ou par `OOMScoreAdjust=` dans une unité systemd. La trace apparaît dans le journal du noyau (« Out of memory: Killed process »).

L'ordonnanceur

L'ordonnanceur décide quel processus s'exécute sur quel cœur et pendant combien de temps. Depuis le noyau 6.6, l'algorithme par défaut est **EEVDF** (Earliest Eligible Virtual Deadline First), qui a remplacé CFS : il répartit équitablement le temps processeur tout en favorisant les tâches sensibles à la latence. EL10, basé sur 6.12, en hérite. La priorité classique (`nice`, de -20 à 19) pondère toujours la part de chacun ; les classes temps réel (`chrt`) passent avant tout.

cgroups v2 : mesurer et limiter

Les **control groups** regroupent des processus pour mesurer et limiter leurs ressources : processeur, mémoire, entrées-sorties, nombre de processus. EL9 et EL10 utilisent la hiérarchie unifiée **cgroups v2**, montée sur `/sys/fs/cgroup`. systemd la structure en *slices* : `system.slice` pour les services, `user.slice` pour les sessions, `machine.slice` pour les machines virtuelles et conteneurs.

```
observer

# arbre des cgroups : chaque processus rattaché à son service ou sa session
systemd-cgls
systemd-cgtop                          # consommation en temps réel par cgroup
systemctl set-property httpd.service MemoryMax=2G CPUQuota=150%   # limite persistante
```

Namespaces : la base des conteneurs

Un **namespace** donne à un groupe de processus une vue privée d'une ressource : ses propres montages (mnt), ses PID (pid), sa pile réseau (net), son nom d'hôte (uts), ses IPC, ses utilisateurs (user, où l'on peut être « root » sans l'être sur l'hôte), son cgroup et son horloge (time). Un conteneur n'est rien d'autre qu'un processus ordinaire placé dans un ensemble de namespaces, limité par des cgroups et confiné par SELinux et seccomp (chapitre 15).

```
observer

lsns                                   # namespaces existants et leur processus initial
# exécuter une commande dans le namespace réseau d'un processus
nsenter -t PID -n ss -tunap
```

À retenir

- fork + exec : copie, puis remplacement du programme ; le PID est conservé.
- loginuid identifie l'humain à l'origine de la session, même après sudo.
- `/proc/PID/exe` et `maps` disent la vérité ; `cmdline` peut mentir.
- Mémoire : regarder « available » ; l'OOM killer laisse une trace dans le journal du noyau.
- Conteneur = processus + namespaces + cgroups + SELinux + seccomp.

6. Le noyau : modules, paramètres et observation

> Le noyau RHEL est un noyau monolithique modulaire : un cœur toujours présent, et des centaines de pilotes chargés à la demande, tous signés.

Les modules

Les modules se trouvent dans `/usr/lib/modules/$(uname -r)/`. Quand udev détecte un périphérique, il demande au noyau de charger le pilote correspondant grâce aux alias déclarés par les modules. `modprobe` résout automatiquement les dépendances (fichier `modules.dep`), contrairement à `insmod`.

```
observer

lsmod | head                                  # modules chargés
modinfo -F signer xfs                         # signataire du module
# options et listes noires de l'administrateur
cat /etc/modprobe.d/*.conf
# (exemple) interdire un module
echo "install usb-storage /bin/false" > /etc/modprobe.d/usb.conf
# 0 = noyau « propre » ; sinon, voir la doc des bits
cat /proc/sys/kernel/tainted
```

Pourquoi ? — des modules signés

Un module s'exécute avec tous les privilèges du noyau : c'est l'outil idéal d'un rootkit. Avec Secure Boot actif, le noyau RHEL passe en mode **lockdown** (intégrité) et refuse les modules non signés par une clé de confiance, ainsi que plusieurs accès directs à la mémoire du noyau. Un module tiers non signé « souille » (*taint*) le noyau, ce qui est visible dans `/proc/sys/kernel/tainted` et dans les traces de plantage.

Trois façons de régler le noyau

Mécanisme	Quand c'est appliqué	Exemple
Paramètres de démarrage	Au boot, via GRUB (`grubby --args`)	`audit=1`, `crashkernel=`, `ipv6.disable=1`
sysctl	À chaud, et au démarrage depuis `/usr/lib/sysctl.d`, `/run/sysctl.d` et `/etc/sysctl.d`	`net.ipv4.ip_forward`, `kernel.kptr_restrict`
Options de module	Au chargement, depuis `/etc/modprobe.d/`	`options kvm_intel nested=1`

Sous le capot — sysctl --system

Au démarrage, `systemd-sysctl` lit les fichiers `*.conf` des trois répertoires par ordre alphabétique de nom de fichier ; à nom identique, `/etc` l'emporte. C'est pourquoi on nomme ses fichiers `90-securite.conf` : un numéro élevé passe après ceux de l'éditeur. `sysctl -a` affiche l'état courant ; `/proc/sys/` en est la représentation en fichiers.

Quand le noyau plante : kdump

En cas de panique, un noyau de secours, préchargé dans une zone de mémoire réservée par le paramètre `crashkernel`, démarre et enregistre une image de la mémoire du noyau planté dans `/var/crash/`. C'est cette image qu'analysent le support ou l'outil `crash`. `kdumpctl status` indique si le mécanisme est opérationnel.

Observer le noyau en direct : eBPF

eBPF permet de charger dans le noyau de petits programmes vérifiés (ils ne peuvent ni boucler indéfiniment ni lire n'importe quelle mémoire) qui s'attachent à des événements : appels système, fonctions du noyau, paquets réseau. C'est la technologie qu'utilisent les EDR modernes, et EL10 fournit des outils prêts à l'emploi.

```
eBPF (en root, à des fins d'observation)


dnf install bpftrace bcc-tools
# Chaque exécution de programme sur la machine, en temps réel
bpftrace -e 'tracepoint:syscalls:sys_enter_execve {
  printf("%d %s -> %s\n", pid, comm, str(args->filename)); }'
/usr/share/bcc/tools/opensnoop -n sshd   # fichiers ouverts par sshd
# connexions TCP sortantes, avec le processus à l'origine
/usr/share/bcc/tools/tcpconnect
```

Regard SOC — le noyau comme cible

Les rootkits noyau modernes passent par un module (bloqué par la signature si Secure Boot est actif) ou par eBPF lui-même. À surveiller : chargements de modules (`init_module`, `finit_module` dans l'audit), programmes eBPF chargés (`bpftool prog list`), changement de la valeur `tainted`, et désactivation de `kernel.unprivileged_bpf_disabled`.

À retenir

- Pilotes = modules signés dans `/usr/lib/modules/` ; `modprobe` gère les dépendances.
- Réglages : paramètres de démarrage (grubby), sysctl (`/etc/sysctl.d`), options de modules (`/etc/modprobe.d`).
- Secure Boot active le lockdown du noyau : plus de modules non signés.
- eBPF permet d'observer tout le système en direct… et peut être détourné.

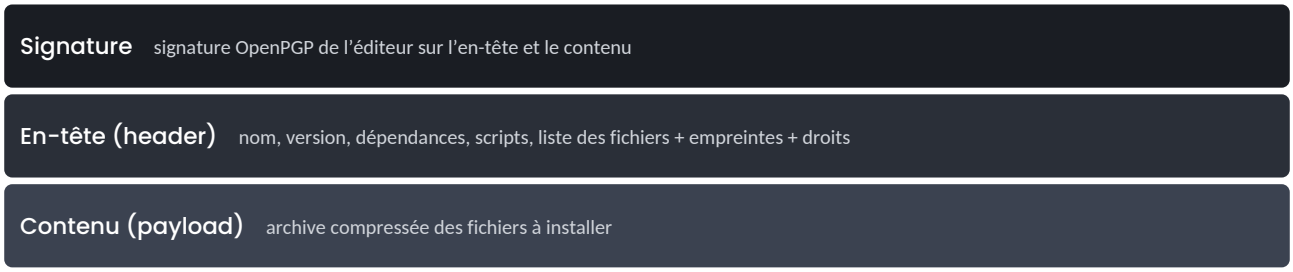
PARTIE 4

Les logiciels : RPM et DNF

Sur un système EL, presque chaque fichier appartient à un paquet, et chaque paquet est signé, versionné et traçable. C'est une mine d'or pour l'administration… et pour l'investigation.

7. RPM : le paquet et sa base de données

> Un RPM n'est pas une simple archive : c'est une archive accompagnée de la description exacte de chaque fichier qu'elle installe (empreinte, droits, propriétaire), que le système conserve pour toujours dans sa base.



Structure d'un fichier .rpm : l'en-tête est recopié dans la base RPM lors de l'installation.

Le nom complet : NEVRA

Chaque paquet est identifié par **N**om, **E**poch, **V**ersion, **R**elease et **A**rchitecture : `openssh-server-9.9p1-7.el10_2.x86_64`. La version est celle du logiciel amont ; la release compte les reconstructions par la distribution, et contient l'étiquette `el10` (la mineure, ici `_2`, apparaît pour les mises à jour spécifiques). L'epoch, rarement visible, est un « joker » qui force l'ordre quand la numérotation amont a changé de logique.

Les scripts d'installation

Un paquet peut contenir des scripts exécutés **en root** à différents moments de la transaction : `%pre` et `%post` (avant/après installation), `%preun` et `%postun` (désinstallation), `%pretrans` et `%posttrans` (début/fin de la transaction complète), ainsi que des « triggers » déclenchés par l'installation d'autres paquets. C'est ainsi qu'un paquet crée un utilisateur système ou recharge systemd.

> **Regard SOC — l'exécution de code à l'installation**
>
> Installer un RPM, c'est exécuter son code en root avant même de lancer le logiciel. Un dépôt tiers non signé ou compromis est donc une porte d'entrée directe. Consultez les scripts d'un paquet avant de l'installer : `rpm -qp --scripts paquet.rpm`.

La base RPM : un inventaire certifié

Depuis EL9, la base RPM est une base SQLite (son emplacement est donné par `rpm --eval '%_dbpath'`). Pour chaque fichier installé, elle mémorise l'empreinte cryptographique, la taille, les droits, le propriétaire, la date et le type (fichier de configuration, documentation…). Cela permet deux opérations précieuses : savoir à quel paquet appartient un fichier, et vérifier qu'un fichier n'a pas été modifié depuis son installation.

```
observer

rpm -qf /usr/sbin/sshd              # à quel paquet appartient ce fichier ?
rpm -ql openssh-server              # fichiers installés par ce paquet
rpm -qc openssh-server              # seulement ses fichiers de configuration
rpm -qi openssh-server              # description, date de construction, signature
rpm -q --changelog openssh-server | head -20
rpm -qa --last | head               # derniers paquets installés ou mis à jour
rpm -V openssh-server               # vérifier l'intégrité des fichiers du paquet
rpm -Va 2>/dev/null | grep -v "^.......T" # tout le système, hors simples changements de date
```

Lire la sortie de rpm -V

Chaque ligne affichée correspond à un fichier qui diffère de ce que la base attend. Les huit premiers caractères indiquent ce qui a changé (un point signifie « identique »), suivis éventuellement d'un `c` pour un fichier de configuration.

Code	Signification	Interprétation
S	Taille différente	Contenu modifié.
M	Mode (droits ou type)	Permissions changées : à examiner (setuid ajouté ?).
5	Empreinte différente	Contenu modifié : **alarmant sur un binaire**, normal sur une configuration.
D	Numéro de périphérique	Rare.
L	Cible de lien symbolique	Lien redirigé : suspect sur un binaire ou une bibliothèque.
U / G	Propriétaire / groupe	Changement de propriété.
T	Date de modification	Souvent bénin seul.
P	Capacités de fichier	Capacité ajoutée ou retirée : à examiner.

> **Exemple**
> `S.5....T. c /etc/ssh/sshd_config` : configuration modifiée, attendu.
> `S.5....T. /usr/bin/ls` : binaire système modifié, **investigation immédiate**.

> **Regard SOC — les limites de rpm -V**
> La vérification compare le disque à la base RPM… qui se trouve sur le même disque. Un attaquant root peut modifier les deux, ou intercepter les appels de `rpm` par un rootkit. Pour une vérification de confiance, analysez le disque depuis un système sain, ou comparez aux empreintes des paquets officiels téléchargés à nouveau. Les fichiers qui n'appartiennent à **aucun** paquet dans `/usr` méritent aussi l'attention.

Les clés de signature

Les paquets officiels sont signés par la clé de la distribution, livrée dans `/etc/pki/rpm-gpg/` et importée dans la base RPM lors de la première utilisation. Une clé importée apparaît comme un pseudo-paquet `gpg-pubkey`.

```
observer

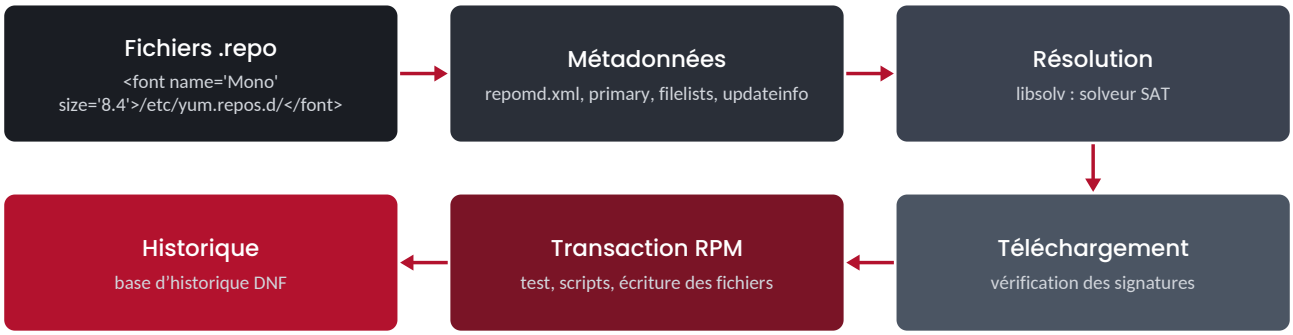
rpm -q gpg-pubkey --qf '%{NAME}-%{VERSION}-%{RELEASE}\t%{SUMMARY}\n'   # clés de confiance
rpm -K paquet.rpm                       # vérifier la signature d'un fichier .rpm
# paquets installés sans signature
rpm -qa --qf '%{NAME}\t%{SIGPGP:pgpsig}\n' | grep -v "Key ID" | head
```

À retenir

- Un RPM = signature + en-tête descriptif + archive ; l'en-tête est conservé dans la base.
- Les scripts `%pre`/`%post` s'exécutent en root à l'installation.
- `rpm -qf` (à qui ?) et `rpm -V` (modifié ?) sont des outils d'investigation de premier ordre.
- Un « 5 » sur un binaire de `/usr` = alerte.

8. DNF : résoudre, télécharger, transacter

rpm sait installer un paquet ; DNF sait lesquels installer, d'où les télécharger et dans quel ordre, en résolvant un problème mathématique de dépendances.



Les métadonnées

Un dépôt est un simple serveur web. À sa racine, `repodata/repomd.xml` référence des fichiers de métadonnées compressés : la liste des paquets et de leurs dépendances (*primary*), la liste des fichiers (*filelists*), les avis de mise à jour (*updateinfo*) et les groupes (*comps*). DNF les met en cache dans `/var/cache/dnf/` et les rafraîchit selon `metadata_expire` (et via `dnf-makecache.timer`).

Pourquoi ? — un solveur SAT

Choisir un ensemble de paquets compatibles entre eux (versions, conflits, « provides » virtuels, architectures) est un problème de satisfiabilité booléenne, réputé difficile dans le cas général. DNF délègue ce calcul à **libsolv**, une bibliothèque issue d'openSUSE, extrêmement rapide. C'est ce qui distingue DNF de l'ancien yum, dont la résolution était lente et parfois incohérente.

La confiance dans un dépôt

Paramètre .repo	Ce qu'il protège
`gpgcheck=1`	Chaque paquet doit être signé par une clé de confiance (`gpgkey=`). **Indispensable.**
`repo_gpgcheck`	Signature des métadonnées elles-mêmes ; souvent désactivée, l'intégrité reposant alors sur HTTPS et la signature des paquets.
`sslverify=1`	Vérification du certificat HTTPS du miroir.
`mirrorlist / metalink`	Choix d'un miroir ; un metalink fournit aussi l'empreinte attendue de repomd.xml.

Les avis de sécurité et les mises à jour ciblées

Les métadonnées *updateinfo* associent chaque mise à jour à un avis (RHSA chez Red Hat, RLSA chez Rocky), à un type (sécurité, correctif, amélioration), à une sévérité et aux CVE corrigées. DNF peut donc répondre à « quelles failles critiques restent ouvertes sur cette machine ? ».

```
observer

dnf updateinfo summary                      # synthèse des avis applicables
dnf updateinfo list --security --sec-severity=Critical
# cette CVE est-elle corrigée par un paquet disponible ?
dnf updateinfo info --cve CVE-2026-31431
dnf upgrade --security                      # n'appliquer que les correctifs de sécurité
# un redémarrage est-il nécessaire (noyau, glibc, systemd) ?
dnf needs-restarting -r
dnf needs-restarting -s                     # services à redémarrer après mise à jour
```

Pourquoi ? — trois noyaux sont conservés

Le noyau est un paquet « installonly » : une mise à jour ajoute le nouveau noyau sans retirer l'ancien (`installonly_limit=3` dans `/etc/dnf/dnf.conf`). Si le nouveau noyau ne démarre pas, il suffit de choisir le précédent dans GRUB. Un noyau mis à jour n'est utilisé qu'après redémarrage.

L'historique et le retour arrière

```
observer

dnf history list | head                     # transactions : qui, quand, quoi
# détail d'une transaction (paquets, ligne de commande, utilisateur)
dnf history info 42
# annuler une transaction (si les anciennes versions sont disponibles)
dnf history undo 42
# paquets installés explicitement (pas comme dépendances)
dnf repoquery --userinstalled
dnf provides '*/bin/nc'                     # quel paquet fournit ce fichier ?
dnf repoquery --whatrequires openssl-libs --installed | head
```

Regard SOC — DNF comme source d'investigation

L'historique DNF enregistre la ligne de commande et l'utilisateur (loginuid) de chaque transaction ; les journaux `/var/log/dnf.rpm.log` et `/var/log/dnf.log` complètent. Un outil réseau (`nmap`, `socat`, `nc`) ou un compilateur installé hors fenêtre de maintenance est un indicateur classique. Un paquet installé directement par `rpm -i` apparaît dans la base RPM mais pas dans l'historique DNF. Vérifiez aussi les dépôts ajoutés : `grep -l 'gpgcheck=0' /etc/yum.repos.d/*.repo`.

Ce qui a changé — EL10 et DNF

RHEL 10 reste sur DNF 4 (la réécriture DNF 5 est utilisée par Fedora mais pas par RHEL 10), sans modularité. Les commandes `yum` restent disponibles comme alias de `dnf`. Les mises à jour automatiques se configurent avec le paquet `dnf-automatic` et ses timers.

À retenir

- Dépôt = serveur web + métadonnées ; DNF les met en cache et les résout avec libsolv.
- `gpgcheck=1` est non négociable ; un dépôt en `gpgcheck=0` est un risque.
- `dnf updateinfo` répond aux questions de vulnérabilités ; `needs-restarting` aux questions de redémarrage.
- `dnf history` est une source d'investigation : qui a installé quoi, quand, avec quelle commande.

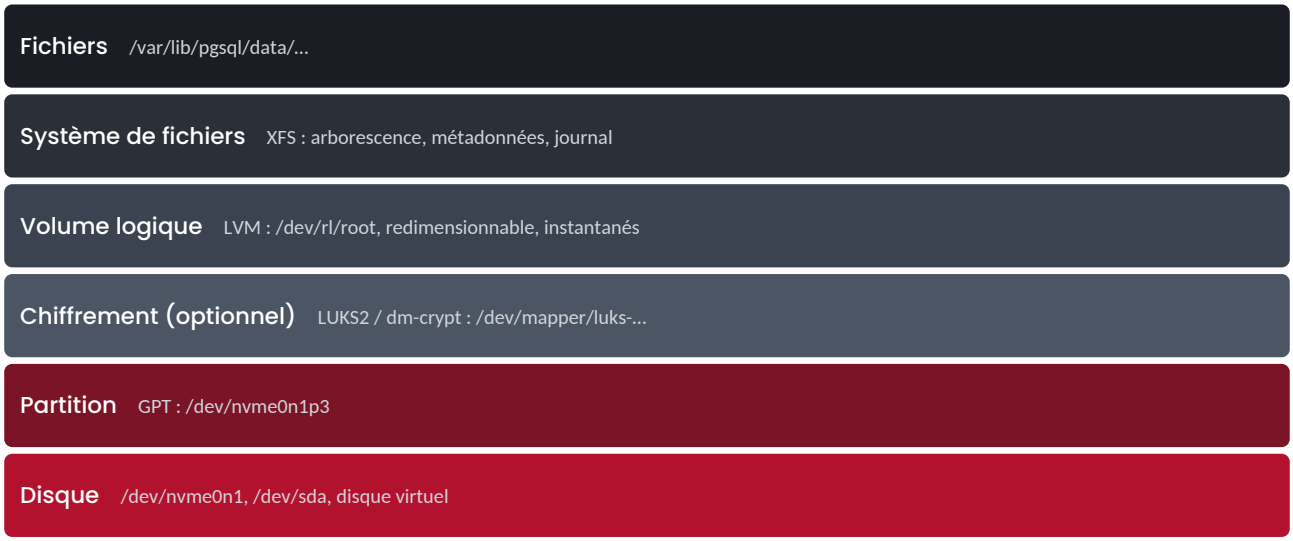
PARTIE 5

Le stockage

Des secteurs physiques jusqu'au fichier que vous ouvrez, les données traversent une pile de couches : partitions, chiffrement, gestion de volumes, système de fichiers. Chaque couche ajoute une fonctionnalité et une raison de panne.

9. Disques, LVM, XFS et montage

> Le stockage EL est un empilement de couches indépendantes : chacune présente au-dessus d'elle un périphérique bloc ordinaire, ce qui permet de les combiner librement.



Chaque couche (sauf la partition) est un périphérique « device-mapper » visible dans /dev/mapper/.

device-mapper : le moteur commun

LVM, LUKS, le RAID logiciel de LVM et le multipath reposent sur un même mécanisme du noyau, le **device-mapper** : une table qui dit « tel secteur du périphérique virtuel correspond à tel secteur de tel périphérique réel, éventuellement transformé (chiffré, dupliqué, copié sur écriture) ». C'est pourquoi `dmsetup ls` et `lsblk` montrent ces couches comme des `dm-N`.

LVM : de la souplesse au-dessus des disques

Volume physique (PV)	Groupe de volumes (VG)	Volume logique (LV)
Un disque ou une partition préparé par pvcreate. Découpé en extents (4 Mio par défaut).	Réservoir d'extents agrégeant un ou plusieurs PV. Peut grandir en ajoutant un PV.	Une « partition virtuelle » prélevée dans le VG. Redimensionnable, instantanés, thin.

Pourquoi ? — LVM

Les partitions sont rigides : agrandir `/var` impose de déplacer tout ce qui suit sur le disque. LVM insère une table de correspondance : un volume logique peut être réparti sur plusieurs disques, agrandi à chaud, déplacé d'un disque à l'autre (`pvmove`) pendant que le système tourne, ou « figé » par un instantané pour une sauvegarde cohérente. Les volumes **thin** allouent l'espace seulement quand il est écrit.

```
observer

# arbre des périphériques, types de systèmes de fichiers, UUID, points de montage
lsblk -f
pvs; vgs; lvs -a                # état de LVM
# agrandir un LV ET son système de fichiers (-r) en une commande
lvextend -r -L +10G /dev/rl/var
```

XFS : le système de fichiers par défaut

XFS est le système de fichiers par défaut d'EL depuis RHEL 7. Conçu pour les gros volumes et le parallélisme, il divise le volume en **groupes d'allocation** indépendants que plusieurs cœurs peuvent manipuler simultanément, retarde l'allocation des blocs jusqu'à l'écriture effective (ce qui réduit la fragmentation), et journalise ses métadonnées pour se remettre rapidement d'une coupure.

Caractéristique	Conséquence pratique
Journalisation des métadonnées	Après une coupure, la structure est cohérente en quelques secondes ; les données en cours d'écriture non synchronisées peuvent être perdues.
Agrandissement à chaud	`xfs_growfs /point` (ou `lvextend -r`).
Pas de réduction	Un XFS ne peut pas rétrécir : il faut sauvegarder, recréer, restaurer. Dimensionnez modestement et agrandissez au besoin.
Reflink	`cp --reflink` copie un fichier instantanément en partageant les blocs jusqu'à modification.
Réparation	`xfs_repair` sur un volume démonté (pas de fsck classique).

Du fstab au montage

`/etc/fstab` liste les montages permanents, identifiés de préférence par UUID (stable même si le disque change de nom). Comme vu au chapitre 4, systemd le convertit en unités `.mount`. Des options utiles : `nofail` (ne pas bloquer le démarrage si le disque est absent), `x-systemd.automount` (monter au premier accès), et pour la sécurité `nodev,nosuid,noexec` sur `/tmp`, `/var/tmp` et `/dev/shm`.

Chiffrement : LUKS2, Clevis et Tang

LUKS2 chiffre un périphérique bloc. La clé maîtresse du volume est stockée dans l'en-tête, chiffrée dans plusieurs « emplacements » (keyslots) : une phrase de passe, un fichier de clé, un TPM… Changer de phrase de passe ne rechiffre pas le disque : on réécrit seulement un emplacement. Pour déverrouiller des serveurs sans saisie, **Clevis** peut lier le volume au TPM2 de la machine ou à un serveur réseau **Tang** (NBDE : le disque ne s'ouvre que s'il démarre dans le bon réseau).

> **Regard SOC — stockage et investigation**
>
> Un instantané LVM permet de figer un volume pour l'acquérir sans arrêter le serveur (`lvcreate -s`). Sur un volume LUKS, une image disque à froid est inutilisable sans la clé : pensez à acquérir la mémoire (où réside la clé maîtresse quand le volume est ouvert) ou à effectuer l'acquisition logique sur le système en marche. Et sauvegardez l'en-tête LUKS (`cryptsetup luksHeaderBackup`) : s'il est détruit, les données sont perdues, ce que certains rançongiciels exploitent.

À retenir

- Pile : disque → partition → (LUKS) → LVM → XFS → fichiers ; tout passe par device-mapper.
- LVM : PV → VG → LV ; `lvextend -r` agrandit volume et système de fichiers.
- XFS s'agrandit à chaud mais ne se réduit jamais.
- Après modification de fstab : `systemctl daemon-reload`.

PARTIE 6

Le réseau

NetworkManager configure les interfaces, le noyau achemine et filtre les paquets, firewalld traduit vos intentions en règles nftables, chrony maintient l'heure exacte. Quatre composants, une seule pile.

10. Réseau, pare-feu et temps

> On ne configure plus des interfaces : on configure des profils de connexion que NetworkManager applique aux interfaces quand elles apparaissent.

NetworkManager : profils et périphériques

Périphérique (device)	Profil (connection)
Une interface réelle ou virtuelle : enp1s0, bond0, br0. Existe ou non selon le matériel. nmcli device	Un ensemble de réglages : IP, DNS, routes, 802.1X. Fichier keyfile dans /etc/NetworkManager/system-connections/ nmcli connection

Un périphérique peut avoir plusieurs profils (bureau, maison) ; un seul est actif à la fois.

Ce qui a changé — le format keyfile

Depuis EL9, les nouveaux profils sont enregistrés au format *keyfile* (`*.nmconnection`, de type INI, lisible uniquement par root). Avec RHEL 10, la prise en charge des anciens fichiers `ifcfg-*` de `/etc/sysconfig/network-scripts/` a été supprimée, de même que le pilote d'agrégation `teamd` (au profit du bonding) et l'utilitaire `dhclient` (NetworkManager utilise son client DHCP interne).

```
observer


nmcli device status
nmcli connection show
nmcli -f ipv4 connection show "System eth0"           # réglages IPv4 d'un profil
nmcli connection modify "System eth0" ipv4.dns "192.0.2.53" ipv4.ignore-auto-dns yes
nmcli connection up "System eth0"                      # appliquer
ip -br addr; ip route; ip rule                         # ce que le noyau applique réellement
```

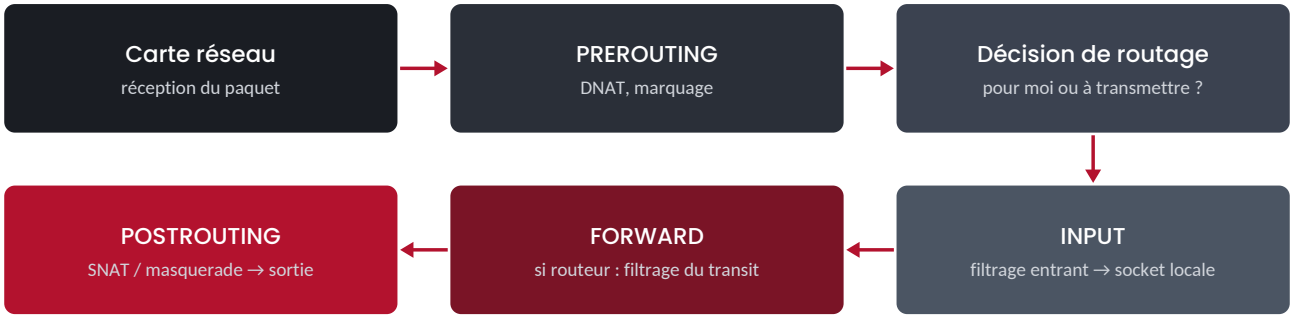
Comment un nom devient une adresse

Quand un programme résout un nom, la glibc consulte `/etc/nsswitch.conf` (ligne `hosts:`) qui fixe l'ordre des sources : d'abord `/etc/hosts` (`files`), puis le DNS, plus des modules comme `myhostname`. Pour le DNS, la glibc lit `/etc/resolv.conf`, que NetworkManager génère à partir des profils actifs. Modifier ce fichier à la main est inutile : il sera réécrit.

Regard SOC — détournement de résolution

Une entrée ajoutée dans `/etc/hosts` ou un serveur DNS modifié dans un profil NetworkManager permet de rediriger silencieusement des mises à jour, un SIEM ou un EDR vers une adresse contrôlée par l'attaquant. Surveillez les modifications de `/etc/hosts`, `/etc/nsswitch.conf` et de `/etc/NetworkManager/system-connections/`.

Le chemin d'un paquet dans le noyau



Les « hooks » netfilter, où nftables attache ses chaînes. Le trafic émis localement passe par OUTPUT puis POSTROUTING.

firewalld et nftables

Le filtrage est fait par **netfilter** dans le noyau, programmé avec **nftables**. firewalld est une surcouche qui raisonne en **zones** (niveaux de confiance) et en **services** (ensembles de ports nommés). Chaque interface ou réseau source appartient à une zone ; chaque zone autorise une liste de services. firewalld génère ensuite les règles nftables correspondantes.

> **Pourquoi ? — runtime et permanent**
> firewalld distingue la configuration en cours d'exécution (immédiate, perdue au rechargement) et la configuration permanente (enregistrée dans `/etc/firewalld/`, appliquée au prochain rechargement). Cela permet de tester une règle sans risque, puis de la rendre persistante. D'où le duo : `firewall-cmd --add-service=https` puis `firewall-cmd --runtime-to-permanent`, ou `--permanent` suivi de `--reload`.

```
observer

firewall-cmd --get-active-zones
firewall-cmd --list-all --zone=public
firewall-cmd --permanent --add-service=https && firewall-cmd --reload
nft list ruleset | less          # la vérité : les règles réellement chargées dans le noyau
ss -tulpn                        # qui écoute, sur quel port, quel processus
```

Le temps : chrony

`chronyd` synchronise l'horloge sur des serveurs NTP, en ajustant progressivement sa vitesse plutôt qu'en la faisant sauter, pour ne pas perturber les applications. `chronyc tracking` indique l'écart et la source ; `chronyc sources -v` liste les serveurs.

> **Regard SOC — l'heure, fondation de l'investigation**
> Sans horloges synchronisées, aucune corrélation n'est possible entre un serveur, un pare-feu et un SIEM. Un écart d'horloge, un changement de source NTP ou une modification de l'heure (`clock_settime`, `adjtimex` dans l'audit) est à la fois un problème opérationnel et un possible anti-forensique. Notez toujours le fuseau des horodatages que vous exportez.

À retenir

- NetworkManager : profils (keyfile) appliqués à des périphériques ; plus d'ifcfg en EL10.
- Résolution : nsswitch.conf → /etc/hosts → DNS de resolv.conf (généré par NetworkManager).
- firewalld (zones, services) génère des règles nftables ; `nft list ruleset` montre la réalité.
- Runtime ≠ permanent dans firewalld.
- chrony maintient l'heure : condition de toute corrélation de journaux.

PARTIE 7

Identités, contrôle d'accès et traçabilité

Qui êtes-vous, qu'avez-vous le droit de faire, et qu'avez-vous fait ? Quatre mécanismes y répondent : NSS et PAM pour l'identité, les permissions et SELinux pour l'autorisation, les politiques cryptographiques pour les échanges, et le sous-système d'audit pour la trace.

11. Utilisateurs, NSS, PAM et sudo

> Linux sépare deux questions : « qui est cet utilisateur ? » (NSS, l'annuaire) et « peut-il se connecter ? » (PAM, l'authentification). C'est ce qui permet de brancher LDAP ou Active Directory sans modifier aucun programme.

Les fichiers de base

Fichier	Contenu	Lisible par
`/etc/passwd`	nom:x:UID:GID:commentaire:répertoire:shell. Le « x » renvoie au fichier shadow.	Tous
`/etc/shadow`	Empreinte du mot de passe, dates d'expiration et de changement.	root uniquement
`/etc/group`	Groupes et membres secondaires (ex. `wheel`).	Tous
`/etc/login.defs`	Valeurs par défaut : plages d'UID, algorithme d'empreinte (`ENCRYPT_METHOD`).	Tous

Le préfixe de l'empreinte dans `/etc/shadow` indique l'algorithme : `$6$` pour SHA-512 itéré, `$y$` pour yescrypt. Un `!` ou `*` en tête signifie que la connexion par mot de passe est verrouillée. Les UID inférieurs à 1000 sont réservés aux comptes système.

NSS : l'annuaire

`/etc/nsswitch.conf` indique, pour chaque base (`passwd`, `group`, `hosts`…), dans quel ordre interroger les sources : `files` (fichiers locaux), `sss` (SSSD : LDAP, FreeIPA, Active Directory), `systemd` (utilisateurs dynamiques). `getent passwd alice` interroge toute la chaîne, contrairement à `grep /etc/passwd`.

PAM : l'authentification modulaire

Un programme qui authentifie (sshd, login, sudo, su, gdm, cockpit) ne vérifie pas lui-même les mots de passe : il appelle la bibliothèque PAM, qui lit `/etc/pam.d/<service>` et exécute une **pile de modules** organisée en quatre étapes.

Étape	Question	Modules typiques
auth	Prouvez-vous votre identité ?	`pam_unix`, `pam_sss`, `pam_faillock` (verrouillage)
account	Votre compte est-il autorisé maintenant ?	expiration, `pam_access`, horaires
password	Comment changer le secret ?	`pam_pwquality` (robustesse), `pam_pwhistory`
session	Que préparer pour la session ?	`pam_loginuid`, `pam_limits`, `pam_systemd`, `pam_selinux`

Sous le capot — les indicateurs de contrôle

Chaque ligne porte un indicateur : **required** (doit réussir, mais la pile continue pour ne pas révéler quel module a échoué), **requisite** (doit réussir, sinon arrêt immédiat), **sufficient** (si réussite et aucun échec « required » antérieur, succès immédiat), **optional**. La syntaxe moderne entre crochets permet un contrôle plus fin.

authselect : ne pas éditer system-auth à la main

Sur EL, `/etc/pam.d/system-auth`, `password-auth` et `nsswitch.conf` sont générés par **authselect** à partir de profils (`local`, `sssd`, `winbind`) et de fonctionnalités (`with-faillock`, `with-mkhomedir`…). Une modification manuelle sera écrasée ; utilisez `authselect select sssd with-faillock` ou un profil personnalisé.

sudo et le groupe wheel

sudo lit `/etc/sudoers` et `/etc/sudoers.d/` (à éditer avec `visudo`, qui vérifie la syntaxe). Par défaut sur EL, les membres du groupe `wheel` peuvent tout exécuter avec leur propre mot de passe. Depuis RHEL 10, l'installeur ajoute par défaut l'utilisateur créé aux administrateurs. Chaque utilisation de sudo est journalisée (journal et audit, type `USER_CMD`).

Le concept clé pour un SOC : loginuid

Lors de l'ouverture de session, le module `pam_loginuid` écrit l'UID de l'utilisateur authentifié dans `/proc/self/loginuid`. Cette valeur est héritée par tous les processus enfants et **ne change pas** quand l'utilisateur passe root par sudo ou su. Elle apparaît dans chaque événement d'audit sous le nom **auid**.

Pourquoi c'est essentiel

Dans les journaux, `uid=0` ne dit pas qui a agi : tout le monde est root après sudo. `auid=1001` dit quel compte humain s'est authentifié à l'origine de la chaîne. Un événement avec `auid=unset` (4294967295) provient d'un service système, pas d'une session : une commande interactive avec un auid non défini sur un serveur web mérite une explication.

```
observer

getent passwd alice; id alice
authselect current                    # profil PAM/NSS actif
faillock --user alice                 # échecs d'authentification comptabilisés
sudo -l -U alice                      # ce qu'alice peut faire avec sudo
cat /proc/self/loginuid               # votre auid, inchangé après "sudo -i"
lastlog; last -n 20; lastb -n 20      # connexions et échecs
```

À retenir

- NSS répond « qui ? » ; PAM répond « autorisé ? » ; les programmes ne font qu'appeler les deux.
- PAM = 4 étapes (auth, account, password, session) gérées par authselect.
- Le préfixe de l'empreinte (6, y) révèle l'algorithme.
- auid (loginuid) suit l'humain à travers sudo : c'est le champ à pivoter en investigation.

12. SELinux : le contrôle d'accès obligatoire

> Les permissions Unix demandent « qui est l'utilisateur ? ». SELinux demande « quel programme est-ce, et a-t-il une raison légitime de toucher cet objet ? ». Même root est soumis à la seconde question.

Pourquoi les permissions classiques ne suffisent pas

Avec le contrôle d'accès discrétionnaire (DAC) classique, un processus a exactement les droits de l'utilisateur qui l'exécute. Si un serveur web tournant sous l'utilisateur `apache` est compromis, l'attaquant peut lire tout ce qu'`apache` peut lire, et si le service tourne en root, tout le système. SELinux ajoute un contrôle d'accès **obligatoire** (MAC) défini par une politique que ni l'utilisateur ni le processus ne peuvent modifier : le serveur web n'a accès qu'aux types d'objets prévus pour un serveur web.

Le contexte : une étiquette sur chaque processus et chaque objet

```
observer

ls -Z /var/www/html/index.html   # system_u:object_r:httpd_sys_content_t:s0
ps -eZ | grep httpd              # system_u:system_r:httpd_t:s0
id -Z                            # unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
#        utilisateur : rôle : TYPE : niveau MLS/MCS
```

Dans la politique « targeted » utilisée par défaut, c'est le **type** qui compte presque toujours. Le type d'un processus s'appelle un **domaine** (`httpd_t`). La politique est un ensemble de règles « allow » de la forme : le domaine `httpd_t` peut lire les fichiers de type `httpd_sys_content_t`. Tout ce qui n'est pas explicitement autorisé est refusé.



SELinux n'intervient qu'après le contrôle classique : il ne peut que restreindre, jamais accorder.

Les transitions de domaine

Comment un processus obtient-il son domaine ? Par **transition** : quand systemd (`init_t`) exécute un fichier de type `httpd_exec_t`, la politique prévoit que le nouveau processus entre dans le domaine `httpd_t`. C'est pourquoi l'étiquette des binaires est cruciale : un exécutable copié ailleurs ou mal étiqueté ne déclenchera pas la bonne transition.

D'où viennent les étiquettes des fichiers

La politique contient des règles de correspondance entre chemins et types (`semanage fcontext -l`). Un nouveau fichier hérite généralement du type de son répertoire parent. `restorecon` réapplique les étiquettes prévues par la politique.

> **Le piège n°1 : mv contre cp**
>
> `cp` crée un nouveau fichier qui prend l'étiquette du répertoire de destination. `mv` déplace le fichier **avec son étiquette d'origine**. Un fichier préparé dans `/home` (`user_home_t`) puis déplacé dans `/var/www/html` sera refusé à Apache. Solution : `restorecon -Rv /var/www/html`.

Adapter la politique sans la désactiver

Besoin	Bon outil	Exemple
Un contenu web dans un répertoire non standard	Règle d'étiquetage	`semanage fcontext -a -t httpd_sys_content_t '/srv/site(/.*)?'` puis `restorecon -Rv /srv/site`
Un service sur un port non standard	Étiquette de port	`semanage port -a -t http_port_t -p tcp 8443`
Un comportement optionnel prévu par la politique	Booléen	`setsebool -P httpd_can_network_connect on`
Un cas vraiment spécifique	Module local, après analyse	`audit2allow -M monmodule` (relire le .te !)
Diagnostiquer un seul domaine	Domaine permissif	`semanage permissive -a httpd_t` (temporaire)

```
observer

getenforce; sestatus                    # mode et politique
ausearch -m AVC,USER_AVC -ts recent     # refus récents
# explication : étiquette, booléen manquant, règle absente
ausearch -m AVC -ts today | audit2why
getsebool -a | grep httpd               # booléens d'un domaine
# désactiver les règles « dontaudit » pour voir les refus masqués
semodule -DB
semodule -B                             # les réactiver ensuite
```

Ce qui a changé — désactiver SELinux

Depuis EL9, il n'est plus possible de désactiver SELinux en écrivant `SELINUX=disabled` dans `/etc/selinux/config` comme autrefois : la méthode documentée est le paramètre de démarrage `selinux=0`. Le mode permissif (`setenforce 0`) reste disponible pour diagnostiquer, mais il supprime toute protection. Dans les deux cas : pour un serveur en production, c'est une décision de sécurité, pas un dépannage.

Regard SOC — SELinux en investigation

Une compromission de service web reste confinée à `httpd_t` : l'attaquant ne peut ni lire `/etc/shadow`, ni écrire dans `/usr/bin`, ni ouvrir une connexion sortante sans le booléen adéquat. Les refus AVC sont donc souvent les **premiers signaux** d'une exploitation : un `httpd_t` qui tente d'exécuter `/bin/sh` ou de se connecter vers l'extérieur. Surveillez aussi les événements `MAC_STATUS` (passage en permissif), `MAC_POLICY_LOAD` et les changements de booléens.

À retenir

- SELinux s'ajoute aux permissions Unix ; il ne peut que restreindre.
- Tout se joue sur le **type** : domaine du processus, type de l'objet, règles allow.
- `mv` conserve l'étiquette ; `restorecon` la corrige.
- Étiquettes, ports et booléens avant tout module personnalisé ; ne jamais désactiver « pour voir ».
- Les refus AVC sont des alertes de sécurité autant que des erreurs de configuration.

13. Cryptographie système et OpenSSH

> Sur EL, on ne règle pas les algorithmes autorisés application par application : une politique cryptographique globale s'impose à toutes les bibliothèques à la fois.

Les politiques cryptographiques

`update-crypto-policies` choisit une politique pour tout le système ; elle est traduite en fichiers de configuration pour chaque bibliothèque (OpenSSL, GnuTLS, NSS, OpenSSH, Kerberos, libssh…) dans `/etc/crypto-policies/back-ends/`. Ainsi, interdire un algorithme faible l'interdit partout d'un coup, au lieu de chercher chaque configuration applicative.

Politique	Usage
DEFAULT	Équilibre sécurité/compatibilité ; adaptée à la plupart des serveurs.
LEGACY	Compatibilité avec des systèmes anciens ; affaiblit la sécurité. Temporaire et justifiée.
FUTURE	Plus stricte (clés plus longues, algorithmes plus forts) ; peut casser des échanges avec des tiers.
FIPS	Restreint aux algorithmes conformes FIPS 140 ; à combiner avec le mode FIPS du système.
Sous-politiques	Ajustements ciblés : `DEFAULT:NO-SHA1`, `DEFAULT:AD-SUPPORT`, modules personnalisés.

Ce qui a changé — le post-quantique

Dans RHEL 10.0, les politiques cryptographiques, OpenSSL, GnuTLS, NSS et OpenSSH prennent en charge des algorithmes post-quantiques en avant-première technologique (Technology Preview). RHEL 10 introduit aussi les outils Sequoia PGP (`sq`, `sqv`) à côté de GnuPG, et fournit OpenSSH 9.9 au lieu de 8.7 dans RHEL 9.

```
observer

update-crypto-policies --show
update-crypto-policies --set DEFAULT:NO-SHA1     # puis redémarrer les services concernés
ls /etc/crypto-policies/back-ends/
# ce que sshd applique vraiment
sshd -T | grep -E "^(ciphers|kexalgorithms|macs|pubkeyacceptedalgorithms)"
```

OpenSSH : configuration et traces

Comme systemd, sshd privilégie les fragments : `/etc/ssh/sshd_config` inclut en premier `/etc/ssh/sshd_config.d/*.conf`. Pour sshd, **la première valeur lue l'emporte** : un fragment nommé `50-redhat.conf` ou `40-local.conf` prend donc le pas sur une ligne du fichier principal. `sshd -T` affiche la configuration effective. Par défaut depuis EL9, root ne peut pas se connecter par mot de passe (`PermitRootLogin prohibit-password`).

Regard SOC — SSH sous surveillance

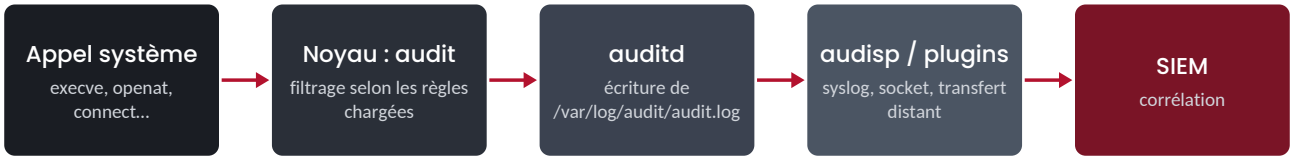
Avec `LogLevel VERBOSE`, sshd journalise l'empreinte de la clé utilisée à chaque connexion : indispensable pour savoir **quelle clé** a servi quand plusieurs personnes partagent un compte. Les fichiers `authorized_keys` (y compris de root et des comptes de service), l'option `AuthorizedKeysCommand`, et les options de clé (`command=`, `from=`) sont des points de persistance classiques.

À retenir

- Une seule commande règle la cryptographie de toutes les bibliothèques : `update-crypto-policies`.
- sshd : fragments dans `sshd_config.d`, première valeur lue gagnante, `sshd -T` pour vérifier.
- EL10 : OpenSSH 9.9, Sequoia PGP, post-quantique en avant-première.

14. Audit et journalisation : la mémoire du système

> Le journal système raconte ce que les programmes veulent bien dire. L'audit du noyau enregistre ce qu'ils font, au niveau des appels système, sans leur demander leur avis.



Les règles

Les règles se placent dans `/etc/audit/rules.d/*.rules` ; `augenrules` les concatène dans l'ordre des noms pour produire `/etc/audit/audit.rules`, chargé au démarrage. `auditctl -l` affiche les règles actives. Deux familles existent : la surveillance de fichiers (`-w`) et les règles d'appels système (`-a`), toutes deux étiquetables par une clé (`-k`) pour faciliter la recherche.

règles d'audit

```
# /etc/audit/rules.d/50-soc.rules  (exemple — à adapter et mesurer en volume)
-w /etc/passwd -p wa -k identite
-w /etc/shadow -p wa -k identite
-w /etc/sudoers -p wa -k privileges
-w /etc/sudoers.d/ -p wa -k privileges
-w /etc/ssh/sshd_config -p wa -k sshd
-w /etc/systemd/system/ -p wa -k persistance
-a always,exit -F arch=b64 -S execve -F euid=0 -F auid>=1000 -F auid!=unset -k exec_root
-a always,exit -F arch=b64 -S init_module,finit_module,delete_module -k modules
-a always,exit -F arch=b64 -S clock_settime,settimeofday,adjtimex -k temps
# -e 2   # à la fin : rend la configuration immuable jusqu'au prochain redémarrage

augenrules --load && auditctl -l
```

Lire un événement

Un événement est composé de plusieurs enregistrements qui partagent le même identifiant `msg=audit(horodatage:numéro)`. Pour une exécution, on trouve typiquement `SYSCALL` (qui, quoi, succès, `auid`, `uid`, `comm`, `exe`), `EXECVE` (les arguments), `CWD`, un ou plusieurs `PATH` et `PROCTITLE` (la ligne de commande, en hexadécimal). `ausearch -i` regroupe et traduit tout cela.

observer

```
ausearch -k identite -i --start today          # événements d'une clé, interprétés
ausearch -m USER_LOGIN,USER_AUTH -sv no -i     # échecs d'authentification
ausearch -ua 1001 -i | less                    # tout ce qu'a fait l'utilisateur d'auid 1001
aureport --summary; aureport -x --summary      # synthèses (exécutables les plus fréquents)
```

journald, rsyslog et l'audit : qui fait quoi

Composant	Source	Forces	Limites
auditd	Noyau (appels système), PAM, SELinux	Exhaustif, fiable, porte l'auid	Volumineux si mal réglé ; format brut
journald	Services, noyau, syslog	Champs fiables (_PID, _UID…), indexé	Peut être volatile ; format binaire
rsyslog	Journal (module imjournal), socket syslog	Fichiers texte, transfert distant (TLS)	Dépend de ce que les programmes écrivent

Sous le capot — sceller le journal

journald peut protéger l'intégrité de ses fichiers par le scellement « Forward Secure Sealing » : `journalctl --setup-keys` génère une clé de vérification à conserver hors de la machine, et `journalctl --verify` détecte ensuite une altération. Cela ne remplace pas l'envoi immédiat vers un collecteur distant, seule vraie garantie face à un attaquant root.

Regard SOC — ce qu'un attaquant root peut faire

Arrêter auditd, supprimer des règles, vider `audit.log`, tronquer le journal. Contre-mesures : règle `-e 2` (configuration immuable, changement seulement après redémarrage, lui-même visible), transfert en temps réel (plugin `audisp` ou rsyslog vers le SIEM), surveillance de l'arrêt d'auditd (`DAEMON_END`, `CONFIG_CHANGE`), et alerte sur toute interruption du flux de journaux d'un hôte. Le silence d'un serveur est un événement.

À retenir

- Audit = ce que font les processus (noyau) ; journal = ce qu'ils disent.
- Règles dans `/etc/audit/rules.d/`, chargées par `augenrules`, clés `-k` pour la recherche.
- `ausearch -i` traduit ; `auid` désigne l'humain ; `-e 2` fige la configuration.
- Seul le transfert immédiat hors de la machine protège les traces d'un attaquant root.

PARTIE 8

Conteneurs, mode image et diagnostic

Les conteneurs assemblent les briques vues jusqu'ici (namespaces, cgroups, SELinux). Le mode image applique la logique des conteneurs au système lui-même. Et quand quelque chose va mal, EL fournit des outils de diagnostic qui méritent d'être connus. Le dernier chapitre assemble tout le guide en suivant une connexion SSH de bout en bout.

15. Podman, conteneurs et mode image

> Podman n'a pas de démon : un conteneur est un processus enfant de votre commande, que vous pouvez confier à systemd comme n'importe quel service.

Modèle avec démon (Docker)	Modèle Podman
Un démon root central exécute tous les conteneurs. Le client parle au démon par une socket. Accès à la socket = root sur l'hôte.	Chaque commande lance directement le conteneur (fork/exec). Rootless : un utilisateur sans privilège peut lancer ses conteneurs. systemd supervise les conteneurs comme des services.

Ce qui se passe lors d'un « podman run »



Ce qui a changé — les composants des conteneurs

En RHEL 10, le runtime `runc` a été retiré au profit de `crun` (écrit en C, plus léger), et le réseau CNI n'est plus disponible : c'est `netavark` qui gère le réseau des conteneurs root. Les conteneurs sans privilège utilisent un réseau en espace utilisateur (pasta). L'exécution de conteneurs RHEL 7 sur un hôte RHEL 10 n'est pas prise en charge.

Rootless et namespaces utilisateur

En mode rootless, Podman crée un namespace utilisateur dans lequel votre UID devient 0. Les UID du conteneur sont projetés sur une plage d'UID subordonnés attribués à votre compte dans `/etc/subuid` et `/etc/subgid`. « root » dans le conteneur n'est donc qu'un utilisateur ordinaire sur l'hôte : une évasion du conteneur ne donne pas root.

SELinux et sVirt : l'isolation entre conteneurs

Chaque conteneur s'exécute dans le domaine `container_t`, avec une paire de catégories MCS aléatoires (`s0:c123,c456`). Ses fichiers portent les mêmes catégories. Deux conteneurs ont le même type mais des catégories différentes : la politique interdit à l'un de toucher les fichiers de l'autre, et à tous de toucher les fichiers de l'hôte. C'est pour cela qu'un volume monté depuis l'hôte nécessite l'option `:Z` (étiquetage privé) ou `:z` (partagé).

Quadlet : les conteneurs comme des services

```
quadlet


# /etc/containers/systemd/web.container  (ou ~/.config/containers/systemd/ en rootless)
[Container]
Image=docker.io/library/nginx:stable
PublishPort=8080:80
Volume=/srv/site:/usr/share/nginx/html:Z
AutoUpdate=registry

[Install]
WantedBy=multi-user.target

# Un générateur systemd transforme ce fichier en web.service :
systemctl daemon-reload && systemctl start web
# UID dans le conteneur, UID réel sur l'hôte, étiquette SELinux
podman ps; podman top web user huser label
```

Le mode image (bootc)

Le **mode image** de RHEL applique au système d'exploitation la logique des conteneurs : le système entier est construit comme une image OCI à partir d'un `Containerfile`, stockée dans un registre, puis déployée et mise à jour de manière **atomique** par `bootc`. `/usr` est en lecture seule, `/etc` est fusionné entre l'image et les modifications locales, `/var` persiste. Une mise à jour prépare la nouvelle image à côté de l'ancienne et bascule au redémarrage ; `bootc rollback` revient à l'état précédent.

Pourquoi ? — un système en image

Sur un parc de centaines de serveurs, chaque machine mise à jour paquet par paquet finit par diverger. Avec une image, toutes les machines exécutent exactement le même `/usr`, testé une fois, versionné, et identifiable par son empreinte. C'est aussi un avantage de sécurité : un binaire système modifié est immédiatement anormal.

Regard SOC — conteneurs et images

Côté détection : un processus dont le cgroup est `machine.slice/libpod-…` appartient à un conteneur ; `podman inspect` et `podman diff` montrent sa configuration et les fichiers modifiés depuis l'image. Points de vigilance : conteneurs lancés avec `--privileged`, montage de `/` ou de la socket de Podman, `--security-opt label=disable`, et images non signées provenant de registres non maîtrisés.

À retenir

- Podman : pas de démon ; conmon surveille, crun crée le conteneur, netavark ou pasta le connecte.
- Rootless : root dans le conteneur = UID subordonné sans privilège sur l'hôte.
- SELinux isole les conteneurs entre eux par catégories MCS ; `:Z` sur les volumes.
- Quadlet transforme un conteneur en service systemd ; bootc transforme le système en image.

16. Diagnostiquer et observer

> Avant de redémarrer un service « pour voir », posez trois questions : que dit le journal, que consomme-t-il, que refuse SELinux ? EL fournit un outil pour chacune.

Question	Outil	Commande de départ
Pourquoi ce service ne démarre-t-il pas ?	systemd + journald	`systemctl status X; journalctl -xeu X`
Est-ce SELinux ?	audit	`ausearch -m AVC -ts recent | audit2why`
Qui consomme le processeur ou la mémoire ?	top, systemd-cgtop	`systemd-cgtop` (par service)
Le disque est-il le goulot ?	iostat, PCP	`iostat -xz 2` (paquet sysstat)
Que fait ce processus ?	strace, perf, eBPF	`strace -fp PID`, `perf top`
Quel profil de performance ?	tuned	`tuned-adm active; tuned-adm recommend`
Historique des performances ?	PCP (Performance Co-Pilot)	`pmlogger` / `pmrep`, graphiques dans Cockpit
Tout collecter pour le support	sos	`sos report` (archive de configuration et de journaux)
Administration graphique	Cockpit	`systemctl enable --now cockpit.socket` puis https://hôte:9090

tuned : des profils plutôt que des réglages épars

`tuned` applique des ensembles cohérents de réglages (sysctl, gouverneur processeur, ordonnanceur d'entrées-sorties) selon l'usage : `throughput-performance`, `latency-performance`, `virtual-guest`, `balanced`… et ajuste dynamiquement certains paramètres. Les profils sont des fichiers lisibles dans `/usr/lib/tuned/`, surchargés dans `/etc/tuned/`.

Red Hat Lightspeed et Insights

RHEL 10 intègre un assistant en ligne de commande, Lightspeed, qui répond en langage naturel aux questions d'administration, et le service Insights qui analyse la configuration des systèmes enregistrés pour signaler risques et vulnérabilités. Ces services relèvent de l'abonnement Red Hat : ils ne sont pas disponibles tels quels sur Rocky Linux.

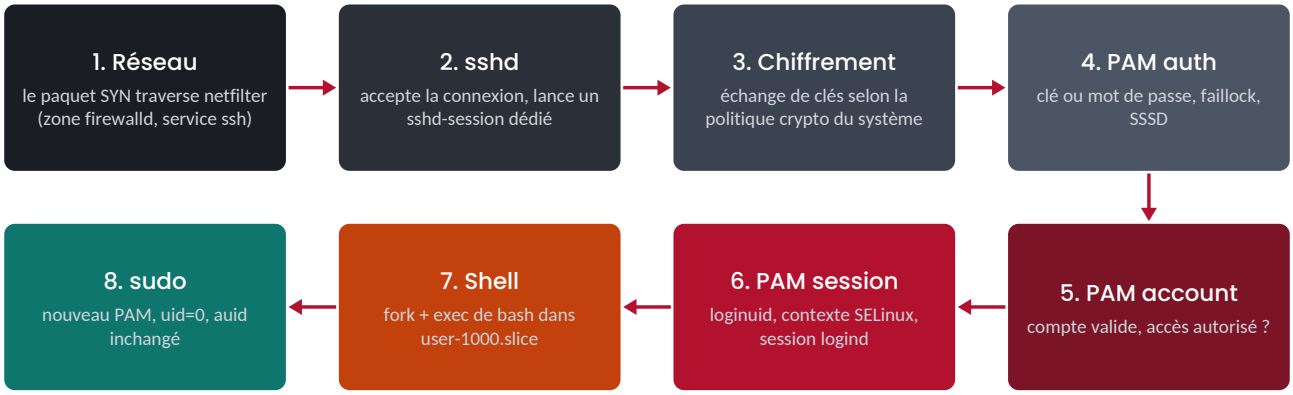
> **Regard SOC — sos report en réponse à incident**
>
> `sos report` collecte en quelques minutes la configuration, les journaux, l'état réseau, les paquets et les services : une excellente « photographie » de triage. Attention : l'archive contient des données sensibles (configurations, parfois des secrets) et la commande modifie peu la machine, mais elle s'exécute en root et lit de nombreux fichiers, ce qui altère les dates d'accès. Elle complète, sans la remplacer, une acquisition forensique.

À retenir

- Journal, ressources, SELinux : les trois réflexes avant toute action.
- tuned, PCP, Cockpit et sos sont fournis et méritent d'être utilisés.
- sos report est un outil de triage rapide, pas une acquisition forensique.

17. Tout assembler : anatomie d'une connexion SSH

Une simple commande « ssh analyste@serveur » met en jeu presque tous les mécanismes de ce guide. La suivre de bout en bout est la meilleure façon de vérifier que l'on a compris.



Chaque étape laisse des traces différentes : c'est ce qui permet de reconstituer une session.

Étape	Mécanisme (chapitre)	Ce que l'on peut observer
Réseau	netfilter, firewalld (10)	`nft list ruleset`, compteurs, journaux de rejet si activés
Acceptation	systemd, processus (4, 5)	sshd écoute ; depuis OpenSSH 9.8, chaque connexion est traitée par un processus `sshd-session` distinct, l'authentification se déroulant dans un processus non privilégié
Chiffrement	crypto-policies (13)	`ssh -vv` côté client : algorithmes négociés ; `sshd -T` côté serveur
Authentification	PAM, SSSD, faillock (11)	audit `USER_AUTH`, `USER_ACCT` ; `/var/log/secure` ; empreinte de clé avec `LogLevel VERBOSE`
Ouverture de session	pam_loginuid, pam_selinux, pam_systemd (11, 12)	audit `LOGIN` (auid attribué), `USER_START`, `USER_LOGIN` ; `loginctl list-sessions`
Processus	fork/exec, cgroups (5)	`systemd-cgls` : `user.slice/user-1000.slice/session-N.scope`
Commandes	audit execve (14)	règles `execve` : chaque programme lancé avec son auid
Élévation	sudo, PAM (11)	audit `USER_CMD` ; journal `_COMM=sudo` ; auid toujours celui d'analyste
Fermeture	PAM session, logind	audit `USER_END`, `CRED_DISP` ; `last`

```
exercice

# Reconstituer la dernière session SSH d'un utilisateur à partir de l'audit
# heure, adresse source, terminal, succès
ausearch -m USER_LOGIN -ua analyste -i | tail -4
ausearch -ua analyste -ts recent -i | grep -E "type=(EXECVE|USER_CMD)" | head -40
# sessions ouvertes et leurs processus
loginctl list-sessions; loginctl session-status
```

Regard SOC — pourquoi cet exercice compte

Lors d'un incident, la question « qu'a fait cette session ? » se résout en pivotant sur trois identifiants : l'adresse source (réseau), le numéro de session et l'auid (PAM et audit), puis l'arbre des processus (cgroup de la session). Si l'un de ces maillons manque, par exemple un journal volatil ou aucune règle `execve`, la reconstitution s'arrête là. Faire cet exercice à froid sur ses propres serveurs est le meilleur test de sa journalisation.

À retenir

- Une connexion SSH traverse netfilter, systemd, OpenSSH, les politiques crypto, PAM, SELinux, logind et l'audit.
- L'auid attribué à l'ouverture de session relie toutes les actions, y compris après sudo.
- Testez la reconstitution d'une session avant d'en avoir besoin.

ANNEXES

Annexes

Correspondance entre anciennes et nouvelles commandes, carte des artefacts utiles en investigation, et sources pour aller plus loin.

Annexe A — Ce que disent les vieux livres, et ce qu'il faut faire aujourd'hui

Ancienne méthode	Sur EL10	Remarque
`service X start`, `chkconfig`	`systemctl start/enable X`	Les anciennes commandes sont des alias ou ont disparu.
Runlevels 3 et 5	`multi-user.target`, `graphical.target`	`systemctl set-default`
`/etc/rc.local`	Une unité systemd dédiée	rc.local n'est exécuté que s'il est rendu exécutable.
`crontab` **pour tout**	Timers systemd	cron reste disponible (cronie).
`ifconfig`, `netstat`, `route`	`ip addr`, `ss`, `ip route`	net-tools n'est plus installé par défaut.
`ifcfg-eth0`	Profils keyfile via `nmcli`	ifcfg n'est plus pris en charge en EL10.
`network-scripts`	NetworkManager	Supprimé depuis EL9.
`iptables -A …`	`firewall-cmd` ou `nft`	firewalld s'appuie sur nftables.
teamd	bonding	teamd retiré en EL10.
`yum`	`dnf`	`yum` reste un alias.
`dnf module enable`	Paquets versionnés ordinaires	Plus de modularité en EL10.
`ntpd`	`chronyd`	ntpd n'est plus fourni.
Modifier `grub.cfg`	`grubby`	Entrées BLS par noyau.
`SELINUX=disabled`	Paramètre `selinux=0` (à éviter)	Changement depuis EL9.
Xorg, VNC	Wayland + Xwayland, RDP	Serveur Xorg et TigerVNC retirés en EL10.
`docker`	`podman`	Commandes largement compatibles ; pas de démon.

Annexe B — Carte des artefacts pour l'analyste

Question d'investigation	Où regarder sur EL
Qui s'est connecté, d'où ?	`last`, `lastb`, `journalctl -u sshd`, `/var/log/secure`, audit `USER_LOGIN`
Qui a utilisé sudo, pour quoi ?	`journalctl _COMM=sudo`, audit `USER_CMD`, `/var/log/secure`
Qu'a exécuté un utilisateur ?	Audit `execve` filtré par `auid` ; historiques `~/.bash_history` (peu fiables)
Quels comptes ou clés ont été ajoutés ?	`/etc/passwd`, `/etc/shadow`, `/etc/group`, `~/.ssh/authorized_keys`, `/etc/sudoers.d/`
Quelle persistance ?	`systemctl list-unit-files --state=enabled`, `systemctl list-timers`, `/etc/systemd/system/`, `~/.config/systemd/user/`, crontabs, `/etc/profile.d/`, `/etc/ld.so.preload`, règles udev, générateurs
Des binaires ont-ils été modifiés ?	`rpm -Va`, fichiers sans propriétaire RPM dans `/usr`
Quels logiciels ont été installés ?	`dnf history`, `rpm -qa --last`, `/var/log/dnf.rpm.log`
Que fait ce processus ?	`/proc/PID/` (exe, maps, fd, cgroup, loginuid), `ss -tunap`, `lsns`
SELinux a-t-il bloqué quelque chose ?	`ausearch -m AVC`, changements de mode (`MAC_STATUS`)
Le pare-feu a-t-il changé ?	`/etc/firewalld/`, `nft list ruleset`, journal de firewalld
Les journaux ont-ils été altérés ?	Trous dans la chronologie, `journalctl --verify`, arrêt d'auditd, flux vers le SIEM interrompu
Quel noyau, quel démarrage ?	`uname -r`, `journalctl --list-boots`, `/boot/loader/entries/`, `/proc/cmdline`, `mokutil --sb-state`

Annexe C — Sources et lectures complémentaires

Les informations propres aux versions proviennent en priorité de la documentation officielle de Red Hat et du projet Rocky Linux, consultée en octobre 2026. Les mécanismes génériques s'appuient sur les pages de manuel et la documentation des projets amont, qui restent les références les plus à jour.

Red Hat Enterprise Linux 10

Notes de version RHEL 10.0 — vue d'ensemble
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/10.0_release_notes/overview

Considerations in adopting RHEL 10 (différences EL9 → EL10)
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html-single/considerations_in_adopting_rhel_10

Considerations in adopting RHEL 10 — réseau
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/considerations_in_adopting_rhel_10/networking

Documentation RHEL 10 (tous les guides, disponible en français)
https://docs.redhat.com/fr/documentation/red_hat_enterprise_linux/10

Cycle de vie de RHEL
https://access.redhat.com/support/policy/updates/errata/

Rocky Linux

Notes de version et cycle de vie
https://docs.rockylinux.org/release_notes

Documentation Rocky Linux (guides, dont des traductions françaises)
https://docs.rockylinux.org

Annonce Rocky Linux 10.2
https://rockylinux.org/news/rocky-linux-10-2-ga-release

Projets amont et références techniques

Pages de manuel Linux (appels système, /proc, capabilities, namespaces, cgroups)
https://man7.org/linux/man-pages/

Documentation du noyau Linux
https://docs.kernel.org

Pages de manuel systemd
https://www.freedesktop.org/software/systemd/man/

Boot Loader Specification
https://uapi-group.org/specifications/specs/boot_loader_specification/

The SELinux Notebook (référence de SELinux)
https://github.com/SELinuxProject/selinux-notebook

Documentation du sous-système d'audit Linux
https://github.com/linux-audit/audit-documentation

RPM
https://rpm.org

DNF
https://dnf.readthedocs.io

Podman
https://docs.podman.io

bootc (mode image)
https://github.com/bootc-dev/bootc

ANSSI — Recommandations de configuration d'un système GNU/Linux
https://messervices.cyber.gouv.fr/documents-guides/fr_np_linux_configuration-v2.0.pdf

> **Conseil de lecture**
> Pour vérifier un comportement sur votre version exacte, la page de manuel installée sur la machine (`man 5 sshd_config`, `man 7 capabilities`, `man systemd.unit`) fait foi : elle correspond précisément au logiciel installé, là où les articles en ligne mélangent les versions.